

Módulo 0 - Teoría

Curso Spring Boot 2026

Índice

Módulo 0 - Preparación del entorno y primer proyecto Spring Boot	5
Propósito del módulo	5
Baseline técnico de esta edición	5
Punto 0.1 - Instalación y verificación del entorno: JDK, Maven e IDE	6
Objetivos de aprendizaje	6
Bloque 1 - Las tres piezas del entorno Java	6
1.1 La JDK: el kit de desarrollo	6
1.2 JVM, JRE y JDK	6
1.3 Del código fuente a la ejecución	7
1.4 Las herramientas que realmente forman la JDK	7
1.5 Qué es realmente el bytecode	8
1.6 Classpath: dónde busca Java las clases	8
1.7 Biblioteca estándar frente a dependencias externas	8
Pregunta	8
Respuesta razonada	9
Bloque 2 - Maven: construcción y dependencias	9
2.1 El problema que Maven resuelve	9
2.2 Coordenadas Maven	9
2.3 Dependencias transitivas	9
2.4 Ciclo de vida	9
2.5 Maven Wrapper	10
2.6 Repositorio local: ~/.m2/repository	10
2.7 Fases, goals y plugins	10
2.8 POM efectivo y versiones gestionadas	10
2.9 Scope de dependencias	11
2.10 Wrapper frente a Maven global	11
Pregunta	11
Respuesta razonada	11
Bloque 3 - El IDE	11
3.1 Qué aporta un IDE	11
3.2 IntelliJ IDEA, Eclipse y VS Code	12
3.3 El IDE no sustituye a JDK ni a Maven	12
3.4 Índices y modelo del proyecto	12
3.5 Configuración versionada y configuración local	12
3.6 Elegir IDE	12
Pregunta	13
Respuesta razonada	13
Bloque 4 - Versiones recomendadas	13
4.1 Java 17 como nivel mínimo del curso	13
4.1.1 Qué significa que Java 17 sea LTS	13
4.1.2 Distribuciones de JDK	13
Pregunta breve	14
Respuesta	14
4.2 Qué ocurre con una JDK demasiado antigua	14

4.3 Maven	14
Pregunta	14
Respuesta razonada	14
Bloque 5 - Instalación por sistema operativo	14
5.1 Windows	14
5.2 Linux	15
5.3 macOS	15
5.4 Qué significa realmente «instalar» cada pieza	15
5.5 Rutas habituales y cómo no confundirlas	15
5.6 Qué hacer cuando hay varias JDK instaladas	16
5.7 Instalación guiada en Windows	16
5.8 Instalación guiada en Debian/Ubuntu	17
5.9 Instalación guiada en macOS	17
5.10 IDE: instalación no equivale a configuración	17
Pregunta	17
Respuesta razonada	18
Bloque 6 - Variables de entorno	18
6.1 JAVA_HOME	18
6.2 PATH	18
6.3 Diagnóstico	18
6.4 Las variables de entorno se heredan al crear procesos	19
6.5 Diagnóstico cruzado: no confíes en un solo comando	19
Pregunta	19
Respuesta razonada	19
Bloque 7 - Las tres verificaciones esenciales	20
7.1 java -version	20
7.2 javac -version	20
7.3 mvn -version	20
Pregunta	20
Respuesta razonada	20
Resumen del Punto 0.1	20
Punto 0.2 - Creación del proyecto con Spring Initializr	21
Objetivos de aprendizaje	21
Bloque 1 - Qué es Spring Initializr	21
Tres formas de utilizarlo	21
Pregunta	21
Respuesta razonada	21
Bloque 2 - El formulario de Initializr	22
2.1 Project	22
2.2 Language	22
2.3 Spring Boot	22
2.4 Project Metadata	22
2.5 Dependencias iniciales	22
2.6 Cómo leer el formulario como decisiones técnicas	22
2.7 Qué es un starter	23
2.8 Qué contiene el ZIP generado	23
2.9 Tres maneras de llegar al mismo proyecto	23
2.10 Qué significa cada dato de Project Metadata	23
2.11 Dependencias: empezar pequeño es una decisión deliberada	24
Pregunta	24
Respuesta razonada	24
Bloque 3 - El pom.xml	24
3.1 La sección parent	25
3.2 Coordenadas del proyecto	25
3.3 Nivel Java	25
3.4 Dependencias iniciales	25
3.5 Plugin Spring Boot	25

3.6 Un pom.xml completo como mapa del proyecto	26
3.7 Cómo investigar el POM sin modificarlo	27
Pregunta	27
Respuesta razonada	27
Bloque 4 - Importación en el IDE	27
IntelliJ IDEA	27
Eclipse	27
VS Code	27
Pregunta	27
Respuesta razonada	28
Bloque 5 - Primera ejecución	28
Pregunta	28
Respuesta razonada	28
Ampliación conceptual - Spring Initializr como generador reproducible, no como asistente mágico	28
La misma intención puede expresarse por distintos canales	29
Pregunta	29
Respuesta razonada	29
Ampliación conceptual - Qué significa realmente importar un proyecto Maven	29
IntelliJ IDEA	30
Eclipse	30
VS Code	30
Pregunta	30
Respuesta razonada	30
Ampliación conceptual - El POM efectivo y la idea de herencia	30
Dependencia directa y dependencia transitiva	31
Pregunta	31
Respuesta razonada	31
Resumen del Punto 0.2	31
Punto 0.3 - Estructura estándar de un proyecto Spring Boot	31
Objetivos de aprendizaje	31
Bloque 1 - Vista general	31
1.1 Por qué importa el paquete de la clase principal	32
1.2 Fuentes frente a artefactos generados	32
1.3 Paquetes Java y carpetas	32
1.4 Source sets de Maven	33
Pregunta	33
Respuesta razonada	33
Bloque 2 - Archivos de la raíz	33
pom.xml	33
.gitignore	33
Maven Wrapper	33
target/	33
HELP.md y otros archivos opcionales de Initializr	33
El ciclo Maven completo y qué directorio modifica	34
Pregunta	34
Respuesta razonada	34
Bloque 3 - src/main/java y src/main/resources	34
Código Java	34
Recursos	34
Configuración externalizada	34
application.properties, YAML y perfiles	35
Pregunta	35
Respuesta razonada	35
Bloque 4 - src/test/java	35
Pregunta	35
Respuesta razonada	36

4.1 @SpringBootApplication como anotación compuesta	36
4.2 Por qué importa la ubicación de la clase principal	36
Bloque 5 - Primer controlador REST	36
5.1 @RestController	36
5.2 @GetMapping	37
5.3 Flujo de una petición	37
Pregunta	37
Respuesta razonada	37
Ampliación conceptual - Estructura estándar, classpath y empaquetado	37
src/main/java	37
src/main/resources	37
src/test/java	37
src/test/resources	38
target	38
Qué ocurre en package	38
Pregunta	38
Respuesta razonada	38
Ampliación conceptual - Escaneo de componentes y ubicación del paquete raíz	38
Pregunta	39
Respuesta razonada	39
Resumen del Punto 0.3	39

Punto 0.4 - Ciclo de trabajo diario: arrancar, leer logs, parar, depurar, testear y probar

APIs	39
Objetivos de aprendizaje	39
Bloque 1 - Qué ocurre al arrancar Spring Boot	39
Arrancar mediante Maven	40
Arrancar el JAR	40
1.1 Las etapas del arranque con algo más de precisión	40
1.2 Tres formas de ejecutar y qué valida cada una	40
1.3 Argumentos desde línea de comandos	41
Pregunta	41
Respuesta razonada	41
Bloque 2 - Leer los logs	41
Niveles	41
Problemas reconocibles en logs	41
2.1 Anatomía de una línea de log	42
2.2 Cómo leer un stack trace	42
2.3 WARN no significa automáticamente “aplicación rota”	42
Pregunta	42
Respuesta razonada	42
Bloque 3 - Parar correctamente	43
3.1 Parada ordenada frente a terminación forzada	43
3.2 Cómo demostrar que la aplicación realmente se ha parado	43
Pregunta	43
Respuesta razonada	43
Bloque 4 - Depuración paso a paso	44
4.1 Breakpoint	44
4.2 Step Over, Step Into y Step Out	44
4.3 Atajos típicos	44
4.4 Pila de llamadas y contexto actual	44
4.5 Watches y evaluación de expresiones	44
4.6 Breakpoints condicionales	45
4.7 Estrategia de depuración	45
4.8 Herramientas concretas que encontrarás en los IDEs	45
Pregunta	45
Respuesta razonada	45
Bloque 5 - Tests en el ciclo diario	46

Pregunta	46
Respuesta razonada	46
Bloque 6 - curl como instrumento de observación HTTP	46
6.1 Método, URL, cabeceras y cuerpo	47
6.2 Redirecciones	47
6.3 Windows PowerShell	47
6.4 Guardar y silenciar salidas	47
6.5 Flags de curl que conviene reconocer desde el principio	47
Pregunta	47
Respuesta razonada	48
Bloque 7 - El mismo flujo en tres IDEs	48
IntelliJ IDEA	48
Eclipse	48
VS Code	48
IntelliJ IDEA: flujo detallado	48
Eclipse: flujo detallado	48
VS Code: flujo detallado	49
Mismo concepto, distintas etiquetas	49
Arranque: equivalencias operativas	49
Depuración: equivalencias	49
Ejecutar tests desde los tres IDEs	49
Cómo leer un fallo de test	50
Diagnóstico rápido del ciclo diario	50
Pregunta	50
Respuesta razonada	50
Ampliación conceptual - El mismo ciclo de trabajo en cuatro interfaces	50
Construir	50
Ejecutar	51
Depurar	51
Testear	51
Probar HTTP	51
Pregunta	51
Respuesta razonada	51
Resumen del Punto 0.4	51

Síntesis del Módulo 0

52

Módulo 0 - Preparación del entorno y primer proyecto Spring Boot

Propósito del módulo

Antes de escribir una API REST, configurar una base de datos o proteger un endpoint, necesitamos comprender y verificar el entorno sobre el que se ejecutará todo el curso. El objetivo de este módulo no es memorizar botones de un IDE: es entender qué piezas intervienen cuando escribimos, compilamos, ejecutamos, probamos y depuramos una aplicación Java y, después, una aplicación Spring Boot.

Al terminar el módulo deberías poder responder con seguridad a preguntas como estas: ¿qué diferencia hay entre JDK, JRE y JVM?, ¿qué hace Maven que no hace Java?, ¿qué contiene realmente un proyecto generado por Spring Initializr?, ¿por qué @SpringBootApplication es tan importante?, ¿qué ocurre desde que ejecutamos main hasta que Tomcat empieza a escuchar en el puerto 8080?, ¿cómo sabemos si un fallo está en Java, Maven, el IDE, Spring o nuestra propia aplicación?

Baseline técnico de esta edición

Este curso utiliza **Java 17**, **Maven 3.9.x** y **Spring Boot 3.5.16** como baseline reproducible. Spring Boot 3.5.x mantiene Java 17 como versión mínima, lo que permite conservar el enfoque del material

de partida y trabajar con una rama moderna de Spring Boot 3 durante todo el itinerario. El alumno puede tener instalada una JDK superior, pero los ejemplos y el proyecto del curso se compilan con nivel de lenguaje Java 17.

La elección de una versión concreta no significa que sea la única válida. Significa que todos los ejemplos, comandos y resultados del curso comparten un punto de referencia estable. Cuando una tecnología cambia, es más fácil aprender si primero dominamos un entorno conocido y después estudiamos las diferencias.

Punto 0.1 - Instalación y verificación del entorno: JDK, Maven e IDE

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar con precisión qué son la JDK, el JRE y la JVM y cómo se relacionan.
2. Explicar para qué sirve Maven y qué problemas evita.
3. Explicar qué aporta un IDE y qué tareas siguen dependiendo realmente de Java y Maven.
4. Entender el papel de JAVA_HOME y PATH.
5. Interpretar `java -version`, `javac -version` y `mvn -version`.
6. Diagnosticar configuraciones mezcladas o incompletas.
7. Compilar y ejecutar mentalmente el ciclo fuente `.java -> bytecode .class -> JVM`.

Bloque 1 - Las tres piezas del entorno Java

1.1 La JDK: el kit de desarrollo

JDK significa *Java Development Kit*. No es un único programa, sino un conjunto de herramientas, librerías y archivos necesarios para desarrollar software Java.

Entre sus componentes principales se encuentran:

- `javac`: compilador de Java. Traduce código fuente `.java` a `bytecode .class`.
- `java`: lanzador de la máquina virtual. Carga clases y ejecuta su `bytecode`.
- `jar`: crea y manipula archivos JAR.
- `javadoc`: genera documentación a partir de comentarios y declaraciones del código.
- `jshell`: entorno interactivo para evaluar expresiones y pequeños fragmentos Java.
- `jdb`: depurador de línea de comandos.
- La biblioteca estándar: colecciones, entrada/salida, fechas, redes, concurrencia y muchas otras APIs que forman parte de la plataforma Java.

Un error habitual al empezar consiste en pensar que “Java” es una sola aplicación. En realidad, cuando escribes `javac`, `java` o `jar`, estás ejecutando herramientas distintas que pertenecen a la misma instalación de la JDK.

1.2 JVM, JRE y JDK

Los tres conceptos forman capas:

Concepto	Qué contiene	Para qué sirve
JVM	Máquina virtual de Java	Ejecuta <code>bytecode</code>
JRE	JVM + librerías necesarias para ejecutar	Ejecuta aplicaciones ya compiladas
JDK	JRE + compilador y herramientas de desarrollo	Desarrolla, compila, ejecuta, empaqueta y depura

La relación conceptual se puede expresar como:

JDK contiene lo necesario para desarrollar y ejecutar; el entorno de ejecución contiene la JVM y las librerías que la aplicación necesita para funcionar.

En las distribuciones modernas de Java ya no siempre encontrarás un “JRE descargable” separado como ocurría en versiones antiguas. La distinción sigue siendo muy útil conceptualmente: **ejecutar** no es lo mismo que **desarrollar**. Para este curso instalaremos una JDK completa.

1.3 Del código fuente a la ejecución

Supón que tenemos:

```
public class Hola {
    public static void main(String[] args) {
        System.out.println("Hola");
    }
}
```

El ciclo básico es:

1. El programador escribe `Hola.java`.
2. `javac Hola.java` analiza el código, comprueba sintaxis y tipos y genera `Hola.class`.
3. `Hola.class` contiene bytecode, no código máquina específico de Windows, Linux o macOS.
4. `java Hola` arranca la JVM, carga la clase y busca `public static void main(String[] args)`.
5. La JVM interpreta y/o compila dinámicamente el bytecode para la plataforma concreta.

Este modelo explica una de las ideas históricas de Java: compilar una vez a bytecode y ejecutar sobre una JVM compatible en distintas plataformas.

1.4 Las herramientas que realmente forman la JDK

Decir que “la JDK sirve para programar en Java” es correcto, pero demasiado genérico para diagnosticar problemas. Conviene asociar cada herramienta con una fase concreta del trabajo:

Herramienta	Entrada	Salida / efecto	Cuándo la usarás
<code>javac</code>	<code>.java</code>	<code>.class</code>	compilación
<code>java</code>	clase o JAR	proceso JVM	ejecución
<code>jar</code>	clases y recursos	<code>.jar</code>	empaquetado / inspección
<code>javadoc</code>	fuentes documentadas	HTML	documentación API
<code>jshell</code>	expresiones Java	resultado interactivo	experimentación rápida
<code>jdb</code>	proceso/clases	sesión de depuración	diagnóstico de bajo nivel
<code>jps</code>	procesos Java	PID y nombre	localizar JVM activas
<code>jstack</code>	PID	stack traces de hilos	bloqueos y cuelgues
<code>jcmd</code>	PID + comando	información diagnóstica	inspección de una JVM
<code>keytool</code>	almacenes/certificados	claves/certificados	HTTPS y seguridad

No necesitas memorizar todas estas herramientas en M0, pero sí entender que una instalación de JDK es mucho más que el ejecutable `java`.

Una comprobación sencilla es localizar físicamente los ejecutables.

Windows PowerShell:

```
Get-Command java
Get-Command javac
```

Windows CMD:

```
where java
where javac
```

Linux/macOS:

```
which java
which javac
```

El resultado debe apuntar a la instalación que quieres utilizar. Si `java -version` dice 17 pero el primer ejecutable resuelto pertenece a una JDK anterior, hay una incoherencia que puede reaparecer al abrir otra terminal o al cambiar de shell.

1.5 Qué es realmente el bytecode

`javac` no genera código máquina específico para tu procesador. Genera **bytecode Java**, almacenado normalmente en archivos `.class`.

Ese bytecode puede ejecutarse en sistemas operativos distintos siempre que exista una JVM compatible con la versión del bytecode y con las dependencias necesarias.

Cuando aparece:

`UnsupportedClassVersionError`

normalmente significa que el `.class` fue generado para una versión de Java más nueva que la JVM usada para ejecutarlo.

```
Hola.java
|
| javac
v
```

`Hola.class` -----> JVM -----> código nativo ejecutado por la CPU

La JVM puede interpretar bytecode y compilar dinámicamente partes frecuentes mediante JIT. Por eso describir Java simplemente como “interpretado” es una simplificación.

1.6 Classpath: dónde busca Java las clases

Cuando ejecutas:

```
java -cp src HolaMinisterio
```

`-cp` significa `--class-path`: la lista de lugares donde la JVM busca clases y recursos.

En el ejercicio mínimo puede ser una sola carpeta. En una aplicación Spring Boot el classpath contiene muchas clases propias y dependencias; Maven y el plugin de Spring Boot lo preparan para no tener que gestionar manualmente cientos de JAR.

Errores como `ClassNotFoundException` o `NoClassDefFoundError` suelen estar relacionados con lo que estaba —o no— disponible en ese classpath.

1.7 Biblioteca estándar frente a dependencias externas

`String`, `LocalDate`, `ArrayList` o `HttpClient` forman parte de Java o de sus módulos estándar.

`SpringApplication`, `RestController` u `ObjectMapper` pertenecen a bibliotecas externas y llegan mediante dependencias.

Esta diferencia explica por qué puedes usar `LocalDate` sin añadir nada al POM pero necesitas Spring Web para `@RestController`.

Pregunta

¿Podrías compilar y ejecutar Java sólo con la JDK, sin Maven ni IDE? ¿Qué perderías en cada caso?

Respuesta razonada

Sí. La JDK basta para escribir un fichero con cualquier editor, compilarlo con `javac` y ejecutarlo con `java`. Sin Maven perderías la automatización de dependencias, el ciclo de construcción, la ejecución organizada de tests y el empaquetado reproducible. Sin IDE perderías ayudas de productividad como autocompletado, navegación, refactorizaciones, depuración visual y gestión cómoda del proyecto. Ni Maven ni el IDE sustituyen a la JDK: se apoyan en ella.

Bloque 2 - Maven: construcción y dependencias

2.1 El problema que Maven resuelve

Una aplicación real rara vez usa sólo la biblioteca estándar. Necesitará frameworks, drivers, clientes HTTP, librerías de test, utilidades, etc. Si gestionáramos todo manualmente tendríamos que:

1. localizar cada JAR;
2. elegir una versión compatible;
3. descargarlo;
4. incluirlo en el classpath;
5. repetir el proceso para sus dependencias transitivas;
6. conseguir que otro desarrollador reproduzca exactamente el mismo entorno.

Maven cambia el enfoque: en vez de copiar bibliotecas manualmente, **declaramos** qué necesita el proyecto y Maven resuelve cómo obtenerlo.

El archivo central es `pom.xml`, *Project Object Model*.

2.2 Coordenadas Maven

Un artefacto Maven se identifica principalmente mediante:

- `groupId`: organización o espacio de nombres;
- `artifactId`: nombre del artefacto;
- `version`: versión concreta.

Por ejemplo:

```
org.springframework.boot:spring-boot-starter-web:3.5.16
```

Estas coordenadas permiten a Maven localizar artefactos en repositorios como Maven Central y almacenarlos en la caché local, normalmente bajo `~/.m2/repository`.

2.3 Dependencias transitivas

Si declaras una dependencia de alto nivel, ésta puede depender de otras. Maven construye un grafo y descarga las necesarias. Gracias a ello, cuando añadamos `spring-boot-starter-web` no tendremos que enumerar manualmente todas las bibliotecas que Spring MVC, Jackson y el servidor embebido necesitan.

Esto no significa que “Maven adivine”. Sigue reglas deterministas de resolución y utiliza la información publicada en los POM de los artefactos.

2.4 Ciclo de vida

Las fases de Maven están ordenadas. Algunas importantes son:

- `validate`: comprueba que el proyecto tiene la estructura básica esperada;
- `compile`: compila código de producción;
- `test`: ejecuta tests;
- `package`: construye el artefacto, por ejemplo un JAR;
- `verify`: ejecuta verificaciones adicionales;
- `install`: instala el artefacto en el repositorio local;

- `deploy`: publica el artefacto en un repositorio remoto configurado.

Cuando ejecutas:

```
mvn package
```

no se ejecuta sólo `package`: Maven recorre las fases anteriores necesarias.

2.5 Maven Wrapper

Un proyecto puede incluir `mvnw`, `mvnw.cmd` y `.mvn/wrapper/`. El objetivo es que el proyecto pueda fijar una distribución concreta de Maven y que el desarrollador no dependa exclusivamente de una instalación global.

En Linux/macOS se utiliza habitualmente:

```
./mvnw test
```

En Windows:

```
mvnw.cmd test
```

El Wrapper mejora la reproducibilidad: dos desarrolladores y un servidor de integración continua pueden ejecutar el mismo proyecto con la misma línea de Maven.

2.6 Repositorio local: `~/.m2/repository`

Maven mantiene una caché local de dependencias. En Linux/macOS suele estar bajo `~/.m2/repository`; en Windows, dentro de la carpeta `.m2` del usuario.

Por eso la primera compilación puede tardar bastante más que las siguientes.

Si declaras `spring-boot-starter-web`, Maven resuelve esa coordenada y recorre su grafo de dependencias transitivas.

No conviene borrar toda `.m2` como reacción automática ante cualquier error. Es mejor identificar primero qué artefacto o resolución está fallando.

2.7 Fases, goals y plugins

Tres conceptos distintos:

Fase. Un punto del ciclo de vida (`compile`, `test`, `package`, `verify`, `install`).

Plugin. Un conjunto de tareas.

Goal. Una tarea concreta de un plugin.

```
./mvnw spring-boot:run
```

invoca un goal del plugin de Spring Boot.

```
./mvnw package
```

invoca una fase del ciclo de vida.

Comprender esta diferencia evita pensar que todos los comandos Maven son “fases”.

2.8 POM efectivo y versiones gestionadas

El `pom.xml` que escribes se combina con configuración heredada y gestión de dependencias.

Por eso algunas dependencias Spring Boot no necesitan declarar una `<version>` explícita.

Puedes ver el POM resultante con:

```
./mvnw help:effective-pom
```

La salida es extensa, pero demuestra que Maven está trabajando con más configuración de la que aparece escrita directamente en tu archivo.

2.9 Scope de dependencias

Los scopes que más reconocerás son:

- `compile`: disponible al compilar y ejecutar;
- `runtime`: necesaria principalmente en ejecución;
- `test`: sólo para tests;
- `provided`: disponible para compilar, pero aportada por el entorno.

`spring-boot-starter-test` debe permanecer en `test`, porque no forma parte del comportamiento productivo de la aplicación.

2.10 Wrapper frente a Maven global

El Maven Wrapper permite que el proyecto use una distribución conocida de Maven sin depender de la versión global instalada por cada desarrollador. Eso mejora la reproducibilidad entre alumnos, compañeros y CI.

El wrapper no sustituye a la JDK: Maven sigue necesitando Java para funcionar.

Pregunta

¿Qué ventaja tiene que Spring Initializr genere también `mvnw` y `mvnw.cmd`? ¿Qué ocurre si un desarrollador no tiene Maven instalado globalmente?

Respuesta razonada

El Wrapper convierte la versión de Maven en parte de la infraestructura reproducible del proyecto. Si el sistema dispone de Java y de las herramientas necesarias para descargar la distribución la primera vez, el desarrollador puede ejecutar el build mediante `mvnw/mvnw.cmd` aunque no haya instalado mvn globalmente. Además evita que cada miembro del equipo utilice una versión distinta sin darse cuenta.

Bloque 3 - El IDE

3.1 Qué aporta un IDE

Un IDE integra en una sola interfaz muchas herramientas de desarrollo:

- resaltado de sintaxis;
- autocompletado contextual;
- navegación a definiciones y usos;
- detección temprana de errores;
- importación de proyectos Maven;
- ejecución y depuración;
- breakpoints e inspección de variables;
- refactorizaciones;
- integración con Git;
- ejecución de tests;
- terminal incorporada;
- formateo y generación de código.

Es importante entender qué hace realmente el IDE. Cuando pulsas “Run”, el IDE no inventa un segundo lenguaje Java. Configura y lanza procesos utilizando la JDK y el modelo del proyecto. Cuando importa Maven, interpreta el POM y sincroniza dependencias con el sistema de construcción.

3.2 IntelliJ IDEA, Eclipse y VS Code

Los tres entornos son válidos para este curso.

Entorno	Fortalezas	Consideraciones
IntelliJ IDEA	Excelente navegación, refactorización y depuración Java	Algunas funciones Spring avanzadas
Eclipse	Maduro, gratuito, muy extendido en Java empresarial	Interfaz y atajos propios; integración
VS Code	Ligero y multiplataforma	Necesita extensiones Java/Spring par

La recomendación pedagógica es mantener un mismo IDE durante la mayor parte del curso. Cambiar constantemente introduce ruido: menús, atajos, configuración de SDK, integración Maven y depurador cambian, aunque Java y Spring sigan siendo los mismos.

3.3 El IDE no sustituye a JDK ni a Maven

Cuando pulsas **Run**, el IDE termina lanzando una JVM. Cuando “recarga Maven”, está leyendo el `pom.xml` y sincronizando el modelo del proyecto.

Por eso puedes tener configuraciones incoherentes como:

```
Terminal: Java 17
IDE:      Java 21
Maven:    Java 11
```

y obtener errores que parecen contradecirse.

En M0 buscamos que terminal, Maven e IDE estén alineados con Java 17.

3.4 Índices y modelo del proyecto

Los IDEs mantienen índices para ofrecer autocompletado, navegación, búsqueda de usos y refactorización.

Después de importar un proyecto Maven es normal que durante unos instantes aparezcan símbolos sin resolver mientras se descargan dependencias y se construyen índices.

Antes de cambiar imports a ciegas, comprueba:

- que el POM sea válido;
- que Maven haya terminado de resolver;
- que el IDE use la JDK correcta;
- que el proyecto haya sido importado como Maven.

3.5 Configuración versionada y configuración local

Versionamos `pom.xml`, fuentes, recursos y tests.

No dependemos de una preferencia privada del IDE para que el proyecto compile.

Un proyecto profesional debe poder construirse en una terminal limpia y en CI.

3.6 Elegir IDE

No existe una única respuesta correcta.

- IntelliJ ofrece una experiencia Java muy integrada.
- Eclipse es maduro y habitual en entornos Java empresariales.
- VS Code es ligero y obtiene soporte Java/Spring mediante extensiones.

El código y Maven son los mismos; cambian menús, atajos y herramientas visuales.

Pregunta

Si el proyecto compila desde la terminal pero el IDE marca errores, ¿qué sospecharías primero?

Respuesta razonada

Sospecharía una diferencia entre el modelo del proyecto que usa Maven y el que ha importado el IDE: SDK incorrecto, proyecto Maven sin recargar, índice corrupto, dependencias todavía no sincronizadas o nivel de lenguaje distinto. El hecho de que Maven compile es una evidencia fuerte de que el código y el POM son coherentes; habría que revisar la configuración local del IDE antes de cambiar el código a ciegas.

Bloque 4 - Versiones recomendadas

4.1 Java 17 como nivel mínimo del curso

Java 17 es una versión LTS y constituye el mínimo de nuestra baseline Spring Boot 3.5.x. El proyecto declarará:

```
<properties>
  <java.version>17</java.version>
</properties>
```

Esto expresa qué nivel de Java usa el proyecto. Aunque una JDK más nueva pueda compilarlo, el código del curso no dependerá de características posteriores a Java 17 salvo que se indique explícitamente.

4.1.1 Qué significa que Java 17 sea LTS

LTS significa *Long Term Support*. Una versión LTS recibe mantenimiento durante un periodo prolongado por parte de distintos distribuidores y se convierte con frecuencia en una base estable para organizaciones que no quieren migrar la plataforma cada pocos meses.

Para aprender Spring Boot importa más la estabilidad de la baseline que perseguir la versión Java más reciente. Java 17 nos ofrece:

- compatibilidad con Spring Boot 3.x;
- una base moderna del lenguaje;
- disponibilidad amplia en Windows, Linux y macOS;
- una versión suficientemente consolidada para que errores del entorno no se confundan con novedades experimentales.

Tener instalada Java 21 o una versión superior no es necesariamente un problema. El proyecto puede compilarse con una JDK más nueva manteniendo `java.version=17`, siempre que no utilicemos APIs o sintaxis posteriores que rompan esa compatibilidad. En el curso, cuando queramos aprovechar una característica que requiera una versión superior, lo indicaremos de forma explícita.

4.1.2 Distribuciones de JDK

«Java 17» no identifica a un único instalador. Existen varias distribuciones compatibles construidas a partir de OpenJDK.

Distribución	Rasgo práctico	Situación habitual
Eclipse Temurin	gratuita, multiplataforma, muy extendida	opción recomendada para homogeneidad
OpenJDK de la distribución Linux	integrada con el gestor de paquetes	muy cómoda en Linux
Oracle JDK	distribución de Oracle	organizaciones que ya trabajan con Oracle
Amazon Corretto	gratuita y mantenida por Amazon	frecuente en entornos AWS

El código Java no debería depender de que el proveedor sea Temurin, Oracle o Corretto. Lo que sí debemos evitar es mezclar ejecutables de proveedores/versiones distintos sin saberlo. Si `java`, `javac`, Maven y el IDE apuntan a instalaciones diferentes, los problemas se vuelven difíciles de reproducir.

Pregunta breve

Si dos alumnos usan Java 17 pero uno tiene Temurin y otro Corretto, ¿debería cambiar el código del curso?

Respuesta

No. Ambas distribuciones implementan la plataforma Java compatible. Puede haber diferencias operativas o de empaquetado, pero el código del curso no debe depender del proveedor. Lo importante es mantener una versión compatible y saber qué instalación utiliza cada herramienta.

4.2 Qué ocurre con una JDK demasiado antigua

Si intentas construir una aplicación moderna con una JVM o un compilador que no soportan las versiones de bytecode requeridas, aparecen mensajes como:

- `UnsupportedClassVersionError;`
- `Unsupported class file major version ...;`
- errores al cargar plugins de Maven;
- fallos de compilación porque el release solicitado no existe.

La dificultad es que el síntoma puede aparecer lejos de la causa. Por eso `java -version`, `javac -version` y `mvn -version` son herramientas de diagnóstico, no simples comprobaciones ceremoniales.

4.3 Maven

Aunque Spring Boot 3.5.x soporta Maven desde versiones anteriores a 3.9, en el curso utilizaremos Maven 3.9.x para disponer de una línea moderna y homogénea. El Wrapper del proyecto fija una distribución 3.9.x, de modo que el alumno no tenga que perseguir diferencias de versión entre equipos.

Pregunta

¿Qué pasaría si instalas Java 8 y después intentas compilar este proyecto? ¿Por qué el error puede resultar confuso?

Respuesta razonada

Java 8 no puede cumplir el nivel mínimo del proyecto ni cargar clases compiladas para Java 17. El error puede aparecer al iniciar Maven, al ejecutar un plugin o al cargar una dependencia, no necesariamente en una línea de nuestro código. Por eso el mensaje puede hablar de versiones de clase o del plugin y hacer pensar que el problema está en Spring cuando la raíz es la JVM utilizada.

Bloque 5 - Instalación por sistema operativo

5.1 Windows

En Windows es habitual instalar una distribución JDK mediante un instalador y Maven mediante un ZIP o un gestor de paquetes.

Para una JDK basada en Temurin, el instalador suele ofrecer opciones equivalentes a:

- configurar `JAVA_HOME`;
- añadir Java al `PATH`.

Si se usan opciones gráficas, conviene verificar después desde una terminal nueva; las terminales abiertas antes del cambio pueden conservar el entorno antiguo.

Maven, si se instala manualmente, se descomprime en una ruta estable y su directorio `bin` debe estar accesible desde `PATH`.

5.2 Linux

En Debian/Ubuntu, una opción habitual es:

```
sudo apt update
sudo apt install openjdk-17-jdk
sudo apt install maven
```

El sistema de alternativas permite comprobar qué ejecutable Java está activo:

```
update-alternatives --config java
```

En Fedora/RHEL se utilizan normalmente paquetes `java-17-openjdk-devel`; en Arch, paquetes equivalentes a `jdk17-openjdk`. Los nombres concretos dependen de la distribución.

5.3 macOS

Homebrew permite gestionar JDK, Maven y aplicaciones de desarrollo desde un sistema común. Por ejemplo:

```
brew install --cask temurin@17
brew install maven
```

macOS incorpora además:

```
/usr/libexec/java_home -v 17
```

que puede ayudar a localizar la ruta de una JDK instalada.

5.4 Qué significa realmente «instalar» cada pieza

Las tres herramientas del entorno no se instalan del mismo modo y conviene entender esa diferencia, porque muchos fallos se producen al asumir que todo funciona como una aplicación gráfica.

La JDK instala un árbol de directorios que contiene ejecutables, módulos, librerías y archivos de configuración. El dato esencial no es que exista una entrada en el menú Inicio, sino que podamos localizar su carpeta raíz y que `java` y `javac` pertenezcan a la misma instalación.

Maven es más sencillo: una distribución binaria es básicamente un directorio con `bin`, `boot`, `conf` y `lib`. Cuando se instala desde un ZIP, no hay un proceso mágico de registro en el sistema. Descomprimir Maven no basta: el shell tiene que poder encontrar `mvn` mediante `PATH`. Por eso dos ordenadores pueden contener exactamente los mismos archivos y, sin embargo, sólo uno reconocer el comando.

El IDE es una aplicación de escritorio, pero internamente mantiene su propio modelo del proyecto. Puede conocer una JDK distinta a la que encuentra la terminal, utilizar el Maven Wrapper del proyecto aunque haya un Maven global instalado y mantener índices que necesitan recargarse cuando cambia el `pom.xml`.

Esta distinción permite diagnosticar con una pregunta muy simple: **¿qué capa está fallando?**

Síntoma	Primera capa que conviene comprobar
<code>java</code> no existe	sistema operativo, <code>PATH</code> , instalación JDK
<code>javac</code> no existe pero <code>java</code> sí	JRE/JDK o <code>PATH</code> mezclado
<code>mvn</code> no existe	instalación Maven o <code>PATH</code>
Maven usa otra Java	<code>JAVA_HOME</code> y configuración de Maven
terminal funciona, IDE no	SDK/JDK configurada en el IDE y modelo Maven
IDE funciona, terminal no	entorno del sistema; el IDE puede estar usando una JDK configurada internamente

5.5 Rutas habituales y cómo no confundirlas

No memorices las rutas como si fueran obligatorias: son ejemplos que te ayudan a reconocer la estructura.

En Windows podrías encontrar:

```
C:\Program Files\Eclipse Adoptium\jdk-17...  
C:\tools\apache-maven-3.9.x
```

En Linux:

```
/usr/lib/jvm/java-17-openjdk-amd64  
/usr/share/maven
```

En macOS:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home  
/opt/homebrew/Cellar/maven/...
```

La regla que sí debes recordar es estructural:

```
JAVA_HOME  
`-- bin  
    |-- java  
    |-- javac
```

Si JAVA_HOME termina en bin, has bajado un nivel de más. Si PATH contiene sólo JAVA_HOME pero no su bin, el shell no encontrará los ejecutables.

5.6 Qué hacer cuando hay varias JDK instaladas

Tener varias JDK no es un problema. Es habitual mantener Java 17 para un proyecto y Java 21 para otro. El problema aparece cuando no sabemos cuál está usando cada herramienta.

En Windows, where java puede mostrar más de una ruta. En Linux/macOS, which java muestra la que resolverá el shell y herramientas como readlink -f \$(which java) ayudan a seguir enlaces simbólicos. En macOS, /usr/libexec/java_home -V enumera las JDK conocidas por el sistema.

Ante una discrepancia, recoge estas cinco evidencias antes de cambiar nada:

1. java -version
2. javac -version
3. mvn -version
4. JAVA_HOME
5. ruta física de java/javac/mvn

Con esas cinco piezas se puede reconstruir casi siempre qué instalación está usando realmente el equipo.

5.7 Instalación guiada en Windows

Una instalación manual típica sigue esta secuencia:

1. descarga una JDK 17 x64/ARM64 acorde con el equipo;
2. instala la JDK en una ruta estable;
3. si el instalador ofrece Set JAVA_HOME, actívalo;
4. si ofrece añadir Java a PATH, actívalo o configúralo manualmente;
5. descarga Maven binario si vas a utilizar también Maven global;
6. descomprime Maven en una carpeta estable;
7. añade ...\\apache-maven-3.9.x\\bin a PATH;
8. cierra las terminales abiertas;
9. abre una terminal nueva y ejecuta java -version, javac -version, mvn -version.

En Windows, JAVA_HOME se puede configurar desde **Variables de entorno**. La variable debe apuntar a la raíz de la JDK, por ejemplo:

```
C:\Program Files\Eclipse Adoptium\jdk-17...
```

Y PATH debe contener una entrada equivalente a:

```
%JAVA_HOME%\bin
```

No introduzcas comillas como parte del valor de JAVA_HOME. Si una herramienta tiene problemas con espacios, se corrige la invocación o configuración concreta; no se cambia arbitrariamente la ruta añadiendo comillas persistentes al valor.

5.8 Instalación guiada en Debian/Ubuntu

El gestor de paquetes puede resolver casi todo el entorno:

```
sudo apt update
sudo apt install openjdk-17-jdk maven
```

Después verifica:

```
java -version
javac -version
mvn -version
```

Si existen varias instalaciones Java:

```
update-alternatives --config java
update-alternatives --config javac
```

No selecciones java y javac de ramas diferentes. Si necesitas definir JAVA_HOME, primero localiza la JDK real en /usr/lib/jvm y apunta a su raíz.

En otras distribuciones cambia el gestor, no el concepto: dnf en Fedora/RHEL, pacman en Arch, etc.

5.9 Instalación guiada en macOS

Con Homebrew:

```
brew install --cask temurin@17
brew install maven
```

Enumera las JDK disponibles:

```
/usr/libexec/java_home -V
```

Y obtiene la ruta de Java 17:

```
/usr/libexec/java_home -v 17
```

Una configuración típica de shell puede derivar JAVA_HOME dinámicamente:

```
export JAVA_HOME=$(/usr/libexec/java_home -v 17)
export PATH="$JAVA_HOME/bin:$PATH"
```

Si lo añades a .zshrc o .bashrc, abre una nueva terminal o recarga el archivo antes de verificar.

5.10 IDE: instalación no equivale a configuración

Después de instalar IntelliJ, Eclipse o VS Code todavía hay que comprobar la JDK que utilizará el proyecto.

- IntelliJ puede tener varios SDK registrados.
- Eclipse distingue JRE/JDK instalados y nivel de compilación.
- VS Code delega gran parte de Java en extensiones y el Java Language Server.

Por eso el cierre de la instalación no es «el icono del IDE aparece en el escritorio», sino «terminal, Maven e IDE coinciden en una plataforma Java compatible».

Pregunta

¿Por qué en Windows es relativamente habitual configurar variables manualmente mientras que en Linux/macOS un gestor de paquetes suele reducir ese trabajo?

Respuesta razonada

Un gestor de paquetes instala programas en ubicaciones conocidas y crea enlaces o registros coherentes con las convenciones del sistema. En Windows, especialmente cuando se descargan instaladores o ZIP desde distintos proveedores, es más frecuente que cada herramienta quede en una ruta independiente y que el usuario tenga que coordinar JAVA_HOME y PATH. No es una limitación de Java, sino una diferencia en los mecanismos habituales de instalación.

Bloque 6 - Variables de entorno

6.1 JAVA_HOME

JAVA_HOME debe representar la carpeta raíz de la JDK que queremos que utilicen herramientas como Maven, Gradle o determinados IDEs.

Ejemplo conceptual en Windows:

```
C:\Program Files\Eclipse Adoptium\jdk-17...
```

Ejemplo en Linux:

```
/usr/lib/jvm/java-17-openjdk-amd64
```

Ejemplo en macOS:

```
/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home
```

No debe apuntar a bin. La carpeta bin vive dentro de JAVA_HOME.

6.2 PATH

PATH es una lista ordenada de directorios donde el sistema busca ejecutables. Cuando escribes:

```
java -version
```

el shell busca un ejecutable llamado java siguiendo el orden configurado.

Por eso pueden producirse situaciones aparentemente contradictorias:

- JAVA_HOME apunta a Java 17;
- pero el primer java de PATH pertenece a Java 11.

Entonces `java -version` y `mvn -version` pueden incluso mostrar JDK diferentes, dependiendo de cómo esté configurado Maven.

6.3 Diagnóstico

En Windows:

```
echo %JAVA_HOME%
where java
where javac
where mvn
```

En PowerShell:

```
echo $env:JAVA_HOME
Get-Command java
Get-Command javac
Get-Command mvn
```

En Linux/macOS:

```
echo "$JAVA_HOME"
which java
which javac
which mvn
```

No basta con saber que un comando existe: queremos saber **qué ejecutable exacto** se ha resuelto.

6.4 Las variables de entorno se heredan al crear procesos

Hay un detalle que explica muchos “lo he cambiado y sigue sin funcionar”.

Cuando abres una terminal o un IDE, ese proceso recibe una copia del entorno disponible en ese momento. Si después cambias `JAVA_HOME`, la terminal que ya estaba abierta puede seguir viendo el valor anterior.

Después de cambiar variables:

1. cierra terminales antiguas;
2. abre una terminal nueva;
3. vuelve a verificar;
4. reinicia el IDE si sigue viendo la configuración anterior.

6.5 Diagnóstico cruzado: no confíes en un solo comando

Si sospechas un conflicto de Java, recoge varias evidencias.

PowerShell:

```
java -version
javac -version
mvn -version
$env:JAVA_HOME
Get-Command java
Get-Command javac
Get-Command mvn
```

CMD:

```
java -version
javac -version
mvn -version
echo %JAVA_HOME%
where java
where javac
where mvn
```

Linux/macOS:

```
java -version
javac -version
mvn -version
echo "$JAVA_HOME"
which java
which javac
which mvn
```

Así separas cuatro preguntas diferentes: qué ejecutable resuelve el shell, qué versión anuncia, qué JDK ve Maven y qué ruta declara `JAVA_HOME`.

Pregunta

Si `java -version` responde “comando no encontrado”, ¿demuestra eso que Java no está instalado?

Respuesta razonada

No. Sólo demuestra que el shell no ha encontrado un ejecutable `java` en las rutas que está consultando. Java podría estar correctamente instalado en otra carpeta y faltar únicamente la entrada correspon-

diente de PATH. Para distinguir ambos casos hay que localizar la instalación y comprobar las variables y rutas configuradas.

Bloque 7 - Las tres verificaciones esenciales

7.1 java -version

Comprueba qué JVM se ejecutará desde esa terminal. Una salida moderna suele indicar:

- versión;
- distribución o proveedor;
- arquitectura;
- información de la VM.

No compila nada. Sólo nos informa del runtime que el shell encuentra.

7.2 javac -version

Comprueba el compilador. Si java funciona pero javac no existe, debemos investigar si estamos usando una instalación incompleta, un PATH incoherente o una distribución que no incluye herramientas de desarrollo en la ruta esperada.

Lo importante es que las versiones sean coherentes.

7.3 mvn -version

Maven muestra algo especialmente útil: además de su propia versión, muestra la versión de Java con la que se está ejecutando y su Java home.

Por tanto, si:

```
java -version -> 17
mvn -version -> Java 11
```

no debemos ignorarlo. Significa que dos herramientas están resolviendo Java de manera distinta.

Pregunta

¿Qué tres comandos ejecutarías siempre al preparar un equipo para este curso y qué te demuestra cada uno?

Respuesta razonada

java -version demuestra qué runtime encuentra el shell; javac -version demuestra que existe un compilador compatible; mvn -version demuestra que Maven está disponible y revela qué Java utiliza Maven. Juntos permiten detectar la mayoría de los problemas de entorno antes de atribuirlos al proyecto.

Resumen del Punto 0.1

- La JDK proporciona compilador, runtime y herramientas de desarrollo.
- La JVM ejecuta bytecode.
- Maven automatiza dependencias y construcción.
- El IDE organiza el trabajo, pero no sustituye a Java ni a Maven.
- JAVA_HOME y PATH cumplen funciones diferentes.
- Las versiones deben ser coherentes entre terminal, Maven e IDE.
- Verificar el entorno antes de programar evita diagnósticos falsos.

Punto 0.2 - Creación del proyecto con Spring Initializr

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar qué es Spring Initializr y qué problema resuelve.
2. Interpretar cada campo importante del formulario.
3. Entender la estructura esencial de un `pom.xml` Spring Boot.
4. Distinguir dependencias directas y transitivas.
5. Importar un proyecto Maven correctamente en un IDE.
6. Entender por qué una aplicación puede arrancar bien y responder 404 en `/`.
7. Explicar el papel del Maven Wrapper y del plugin de Spring Boot.

Bloque 1 - Qué es Spring Initializr

Spring Initializr es un servicio que genera el esqueleto inicial de un proyecto Spring. Puede utilizarse desde su interfaz web, desde integraciones de IDE y mediante API.

No genera la aplicación de negocio. Genera **la infraestructura inicial correcta**: estructura Maven/Gradle, metadatos, dependencias elegidas, clase principal, test básico, configuración vacía o mínima y Wrapper.

Sin una herramienta así tendríamos que crear a mano:

- `src/main/java`;
- `src/main/resources`;
- `src/test/java`;
- el POM;
- el plugin de Spring Boot;
- la clase principal;
- scripts Wrapper;
- `.gitignore`;
- configuración inicial.

Nada de eso es imposible, pero al principio distraería de los conceptos que queremos aprender.

Tres formas de utilizarlo

Web. Entrar en `start.spring.io`, elegir opciones y descargar un ZIP.

IDE. Muchos IDEs incluyen asistentes que llaman al servicio o reproducen sus opciones.

API/automatización. Initializr dispone de endpoints que permiten generar proyectos desde herramientas o scripts. Es útil en automatización, aunque no necesitamos dominarlo para comenzar.

Pregunta

¿Por qué un generador como Spring Initializr mejora la reproducibilidad y no sólo la comodidad?

Respuesta razonada

Porque convierte una serie de decisiones implícitas y manuales en un conjunto de opciones declaradas: versión de Spring Boot, Java, sistema de construcción, coordenadas y dependencias. El resultado puede regenerarse o revisarse con mucha más facilidad que una estructura montada manualmente sin registro de sus decisiones.

Bloque 2 - El formulario de Initializr

2.1 Project

Elegiremos **Maven**. Gradle también es una excelente herramienta, pero mezclar dos sistemas de construcción en un curso introductorio añadiría variabilidad innecesaria.

2.2 Language

Elegiremos **Java**. Spring Boot también admite Kotlin y Groovy, pero el itinerario está diseñado sobre Java.

2.3 Spring Boot

Para esta edición fijamos 3.5.16. Es importante evitar accidentalmente versiones SNAPSHOT o M (milestone) si lo que buscamos es una base estable y reproducible.

2.4 Project Metadata

Usaremos:

```
Group:      es.mecd.demo
Artifact:    mi-proyecto
Name:        mi-proyecto
Description: Proyecto acumulativo del curso Spring Boot 2026
Package:     es.mecd.demo.miproyecto
Packaging:   Jar
Java:        17
```

Group suele representar el espacio de nombres de la organización.

Artifact identifica el artefacto dentro del grupo y normalmente coincide con el directorio/proyecto.

Name es el nombre legible del proyecto.

Package name será la raíz de los paquetes Java y es importante para el escaneo de componentes.

Packaging JAR es apropiado para una aplicación Spring Boot autocontenida con servidor embebido.

2.5 Dependencias iniciales

Comenzamos con **Spring Web**. La regla pedagógica es deliberada: añadir sólo lo que podemos explicar y utilizar.

Spring Web aporta la infraestructura necesaria para construir aplicaciones web Spring MVC y, mediante el starter, incorpora el servidor embebido y otras dependencias coherentes.

El proyecto también incluirá spring-boot-starter-test como dependencia de test.

2.6 Cómo leer el formulario como decisiones técnicas

Los campos de Initializr no son simples datos administrativos.

Group. Suele representar organización o dominio invertido, por ejemplo es.mecd.demo.

Artifact. Identifica el artefacto Maven, por ejemplo mi-proyecto.

Name. Nombre humano del proyecto.

Description. Descripción inicial.

Package name. Paquete Java base. Normalmente allí se crea la clase principal y desde ese paquete Spring empieza el escaneo de componentes.

Packaging. Usamos JAR. El artefacto final podrá ejecutarse con `java -jar`.

Java. Elegimos 17 para mantener un baseline homogéneo.

Una decisión aparentemente pequeña como el package base afecta a la estructura que construiremos en los siguientes módulos.

2.7 Qué es un starter

Un *starter* expresa una capacidad de alto nivel.

Seleccionar **Spring Web** añade `spring-boot-starter-web`, que arrastra un conjunto coherente de dependencias para aplicaciones web.

La idea es declarar intención:

“quiero una aplicación web”

en lugar de mantener manualmente una lista de bibliotecas y versiones compatibles.

2.8 Qué contiene el ZIP generado

Initializr produce un proyecto, no una carpeta llena de todas las dependencias binarias.

El ZIP incluye estructura, POM, wrapper, clase principal, configuración y test inicial. Maven descargará las dependencias cuando el proyecto se importe o compile.

Por eso el ZIP puede ser pequeño y la primera sincronización Maven tardar más.

2.9 Tres maneras de llegar al mismo proyecto

Spring Initializr no es únicamente una página web. El servicio genera la misma clase de proyecto desde distintos clientes.

Desde la web, completas el formulario y descargas un ZIP. Es la opción más transparente cuando estás aprendiendo porque puedes ver todas las decisiones.

Desde un IDE, IntelliJ, Eclipse con Spring Tools o VS Code pueden presentar un asistente propio. El IDE envía las opciones al servicio de Initializr y abre el proyecto resultante. Es cómodo, pero es importante entender que el IDE no está inventando una estructura diferente.

Desde una petición HTTP, Initializr expone endpoints que permiten automatizar la generación. No necesitamos depender de esa forma en el curso, pero entender que existe ayuda a separar el concepto de la interfaz gráfica: Spring Initializr es un servicio de generación de proyectos, no «una pantalla con botones».

La consecuencia pedagógica es importante. Si sabes leer un `pom.xml` y la estructura generada, podrás reconocer un proyecto aunque haya sido creado desde otra interfaz.

2.10 Qué significa cada dato de Project Metadata

Los campos del formulario no son decoración. Se convierten en nombres que después aparecen en Maven, Java y el sistema de archivos.

Campo	Ejemplo del curso	Efecto principal
Group	<code>es.mecd.demo</code>	identidad organizativa y base habitual del paquete
Artifact	<code>mi-proyecto</code>	nombre Maven del artefacto y del JAR
Name	<code>mi-proyecto</code>	nombre legible del proyecto
Description	texto descriptivo	metadato humano
Package name	<code>es.mecd.demo.miproyecto</code>	paquete de la clase principal
Packaging	<code>Jar</code>	tipo de artefacto que construirá Maven
Java	<code>17</code>	nivel de lenguaje/bytecode esperado

Hay dos errores frecuentes que parecen pequeños pero se arrastran durante todo el proyecto:

1. elegir un package distinto y después copiar código que presupone otro paquete;
2. cambiar artifactId y no darse cuenta de que cambia también el nombre del JAR final.

2.11 Dependencias: empezar pequeño es una decisión deliberada

En el primer proyecto incorporamos sólo lo que necesitamos. Spring Web aporta Spring MVC, el servidor embebido y la infraestructura necesaria para exponer endpoints HTTP. Spring Boot Starter Test llega normalmente como dependencia de test en el proyecto generado.

No añadimos JPA, Security, Validation, Actuator o bases de datos «por si acaso». Cada starter adicional:

- amplía el classpath;
- puede activar auto-configuraciones;
- añade conceptos que el alumno todavía no necesita;
- puede alterar los logs y el comportamiento de arranque.

Un proyecto pequeño hace visibles las relaciones causa-efecto. Cuando en módulos posteriores añadimos una dependencia, podremos observar qué capacidad nueva aparece por esa decisión.

Pregunta

¿Por qué no conviene marcar diez starters “por si acaso” al crear el primer proyecto?

Respuesta razonada

Porque cada starter modifica el classpath y puede activar auto-configuración adicional. Si añadimos tecnologías que todavía no comprendemos, aparecen beans, logs, configuración y posibles errores sin contexto. Empezar con el conjunto mínimo hace que cada incorporación posterior tenga una causa pedagógica clara.

Bloque 3 - El pom.xml

Un POM de Spring Boot puede incluir:

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.16</version>
  <relativePath/>
</parent>
```

El parent proporciona gestión de versiones y configuración coherente para muchos plugins y dependencias del ecosistema.

Las coordenadas del proyecto:

```
<groupId>es.mecd.demo</groupId>
<artifactId>mi-proyecto</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

La versión de Java:

```
<properties>
  <java.version>17</java.version>
</properties>
```

Dependencias:

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
```

```

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

```

Y el plugin:

```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

```

El plugin permite, entre otras tareas, ejecutar la aplicación desde Maven y reempaquetar el JAR para que pueda arrancarse con `java -jar`.

3.1 La sección parent

Una declaración típica de esta edición es:

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.5.16</version>
  <relativePath/>
</parent>

```

El parent aporta convenciones y gestión de versiones.

3.2 Coordenadas del proyecto

```

<groupId>es.mecd.demo</groupId>
<artifactId>mi-proyecto</artifactId>
<version>0.0.1-SNAPSHOT</version>

```

SNAPSHOT identifica una versión de desarrollo que todavía puede cambiar.

3.3 Nivel Java

```

<properties>
  <java.version>17</java.version>
</properties>

```

Esto declara qué nivel Java espera el proyecto. No instala la JDK ni corrige una JVM antigua.

3.4 Dependencias iniciales

spring-boot-starter-web aporta la capacidad web.

spring-boot-starter-test aporta herramientas de test y queda limitado al scope test.

3.5 Plugin Spring Boot

```

<plugin>
  <groupId>org.springframework.boot</groupId>

```

```
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

Permite, entre otras cosas, crear el JAR ejecutable y usar el goal `spring-boot:run`.

El POM es la receta reproducible de construcción del proyecto.

3.6 Un pom.xml completo como mapa del proyecto

Un POM generado por Initializr para nuestro primer proyecto tiene una forma parecida a ésta. La versión exacta puede variar, pero la función de cada bloque permanece estable:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.5.16</version>
    <relativePath/>
  </parent>

  <groupId>es.mecd.demo</groupId>
  <artifactId>mi-proyecto</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>mi-proyecto</name>
  <description>Primer proyecto Spring Boot del curso</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

No memorices el XML. Aprende a localizar zonas:

- **parent:** gestión y convenciones heredadas de Spring Boot;
- **coordenadas:** quién es nuestro artefacto;
- **properties:** decisiones globales como Java 17;
- **dependencies:** qué necesita la aplicación para compilar/ejecutar/testear;
- **build/plugins:** cómo participa Maven en la construcción.

La ausencia de <version> en muchos starters no significa que Maven «adivine». La versión queda gestionada por el ecosistema de dependencias de Spring Boot. Esa gestión evita que cada alumno combine versiones incompatibles de Spring Framework, Jackson, Tomcat y otras librerías.

3.7 Cómo investigar el POM sin modificarlo

Dos comandos son especialmente útiles cuando algo no encaja:

```
./mvnw dependency:tree
./mvnw help:effective-pom
```

`dependency:tree` muestra la jerarquía de dependencias, incluidas las transitivas. `help:effective-pom` muestra el POM efectivo después de aplicar herencia, propiedades y configuración. El segundo puede ser muy largo; se utiliza como herramienta de diagnóstico, no como archivo que haya que editar.

Pregunta

¿Qué ventaja tiene que el parent gestione versiones? ¿Sería imposible declarar versiones manualmente?

Respuesta razonada

No sería imposible, pero tendríamos que coordinar nosotros muchas combinaciones de versiones entre componentes relacionados. La gestión proporcionada por Spring Boot define un conjunto probado de dependencias. Podemos sobrescribir versiones cuando exista un motivo, pero dejar la gestión por defecto reduce incompatibilidades accidentales.

Bloque 4 - Importación en el IDE

IntelliJ IDEA

Al abrir la carpeta que contiene el `pom.xml`, IntelliJ detecta Maven e importa el modelo. Hay que comprobar:

- Project SDK = JDK 17 o compatible;
- language level coherente;
- Maven sincronizado;
- dependencias resueltas.

Eclipse

El proyecto se importa como **Existing Maven Project**. Después conviene comprobar Installed JREs, Maven y el nivel de compilación.

VS Code

Con *Extension Pack for Java* y *Spring Boot Extension Pack*, abrir la carpeta raíz permite al Java Language Server importar el proyecto Maven. Hay que esperar a que termine la indexación antes de interpretar como reales todos los errores visuales iniciales.

Pregunta

¿Por qué debemos verificar el SDK del IDE incluso si `java -version` en la terminal es correcto?

Respuesta razonada

Porque el IDE puede mantener su propia configuración de SDK por proyecto. La terminal y el IDE son procesos distintos. Es posible que el sistema resuelva Java 17 mientras el proyecto del IDE apunta a otra JDK. La solución no es asumir que ambos entornos son idénticos, sino comprobarlos.

Bloque 5 - Primera ejecución

La clase principal tendrá esta forma:

```
package es.mecd.demo.miproyecto;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MiProyectoApplication {

    public static void main(String[] args) {
        SpringApplication.run(MiProyectoApplication.class, args);
    }
}
```

Cuando la ejecutamos, Spring Boot arranca el contexto y, al existir Spring Web, levanta un servidor HTTP embebido.

Si todavía no hemos creado ningún controlador, visitar:

`http://localhost:8080/`

puede devolver un 404. **Eso no significa que la aplicación no haya arrancado.** Significa que el servidor está funcionando pero no existe un manejador para /.

La evidencia de arranque se obtiene de los logs y de la capacidad del puerto para aceptar conexiones, no de asumir que toda URL debe devolver 200.

Pregunta

¿Por qué un 404 puede ser una señal de que el servidor sí funciona?

Respuesta razonada

Porque para devolver un 404 la petición ha llegado a un servidor que ha procesado la ruta y ha determinado que no existe un recurso o manejador asociado. Un error de conexión, en cambio, indicaría que no hay ningún proceso escuchando o que la red impide alcanzarlo.

Ampliación conceptual - Spring Initializr como generador reproducible, no como asistente mágico

Spring Initializr se utiliza a menudo como si fuese una página que «crea un proyecto». Esa descripción es correcta pero incompleta. Pedagógicamente conviene entenderlo como un **generador de una configuración inicial reproducible**. Recibe una serie de decisiones y produce un conjunto coherente de archivos que expresan esas decisiones en Maven, Java y Spring Boot.

Las decisiones principales son:

- sistema de construcción;
- lenguaje;
- línea de Spring Boot;
- coordenadas Maven;
- nombre y descripción;

- paquete raíz;
- tipo de empaquetado;
- versión de Java;
- starters iniciales.

Cuando pulsas **Generate**, Initializr no crea una aplicación de negocio terminada. Crea una base que ya sabe cómo compilarse, testearse y arrancar. Ese punto de partida es valioso porque evita que cada alumno tenga que recordar de memoria la estructura completa de Maven y Spring Boot antes de haberla aprendido.

La misma intención puede expresarse por distintos canales

La web de Initializr, los asistentes integrados en los IDE y una petición HTTP a `start.spring.io` son interfaces diferentes para la misma idea: enviar metadatos y obtener un proyecto.

Web. Resulta didáctica porque permite ver las opciones de forma explícita y revisar qué se ha seleccionado.

IntelliJ IDEA. Puede ofrecer un asistente de proyecto Spring que recopila los mismos metadatos desde el entorno de trabajo.

Eclipse con Spring Tools. Puede crear un Spring Starter Project mediante un asistente equivalente.

VS Code. El comando Spring Initializr: Create a Maven Project permite realizar la misma operación desde la Command Palette.

Consola/API. Initializr expone un servicio HTTP y por tanto puede automatizarse. Un ejemplo conceptual sería solicitar un `starter.zip` con parámetros. En un entorno de CI o una plantilla corporativa, esta vía puede ser más útil que una interfaz gráfica.

Lo importante es distinguir **canal** de **resultado**. Si los parámetros son equivalentes, el proyecto generado debe expresar las mismas decisiones esenciales. El IDE no añade una semántica distinta a Maven por el hecho de presentar un formulario.

Pregunta

¿Por qué es importante enseñar también la vía de consola si la web es más cómoda para el primer proyecto?

Respuesta razonada

Porque la consola demuestra que el proyecto puede generarse y construirse sin interacción manual. Esa propiedad es fundamental para automatización, CI/CD y reproducibilidad. La web es excelente para aprender los campos; la línea de comandos enseña que esos campos son datos y que el proceso puede automatizarse.

Ampliación conceptual - Qué significa realmente importar un proyecto Maven

«Abrir una carpeta» y «importar un proyecto Maven» pueden parecer la misma operación, pero el IDE debe construir un **modelo del proyecto** a partir de `pom.xml`.

Ese modelo contiene, entre otras cosas:

- versión de Java;
- rutas de código fuente;
- dependencias directas y transitivas;
- plugins;
- configuración de tests;
- recursos;
- classpath de compilación;
- classpath de test.

Cuando modificas `pom.xml`, el modelo del IDE puede quedar temporalmente desactualizado hasta que Maven se recarga. Por eso aparecen situaciones como «Maven compila pero el IDE pinta imports en rojo». No es necesariamente una contradicción en el código; puede ser una divergencia entre el **modelo real que Maven calcula** y el **modelo que el IDE tiene en memoria**.

IntelliJ IDEA

IntelliJ mantiene un modelo interno del proyecto y una integración Maven. El Project SDK determina la JDK principal del proyecto, pero una configuración de ejecución concreta puede especificar otra. Además, el importador Maven puede tener su propia JDK configurada. En un diagnóstico serio conviene saber qué capa estamos mirando.

Eclipse

Eclipse utiliza su modelo de proyecto Java y la integración m2e para Maven. El classpath se refleja mediante contenedores administrados y la configuración de la JRE/JDK puede depender del workspace y del proyecto. Maven > Update Project fuerza una sincronización del modelo Maven.

VS Code

VS Code se apoya en extensiones y en el Java Language Server. La carpeta abierta, el runtime Java configurado y el estado de importación del workspace determinan lo que el editor conoce del proyecto. Si el Language Server no ha terminado de importar, el editor puede mostrar errores transitorios aunque Maven todavía no haya fallado.

Pregunta

¿Qué evidencia pesa más para saber si el proyecto compila: un subrayado rojo del editor o un `./mvnw clean test` terminado con éxito?

Respuesta razonada

El build Maven ejecutado con el wrapper es una evidencia reproducible del estado del proyecto. El editor aporta una señal muy útil, pero puede estar desincronizado. Si Maven pasa y el IDE no, primero se investiga la configuración o sincronización del IDE; no se modifica código correcto para satisfacer un índice local desactualizado.

Ampliación conceptual - El POM efectivo y la idea de herencia

El `pom.xml` que vemos no contiene necesariamente toda la configuración que Maven utiliza. Maven combina el POM del proyecto con:

- POM padre;
- propiedades;
- gestión de dependencias;
- perfiles activos;
- defaults de plugins;
- configuración global o de usuario cuando corresponda.

El resultado es el **POM efectivo**. Por eso:

```
./mvnw help:effective-pom
```

puede mostrar mucho más XML que nuestro archivo original.

En Spring Boot, el parent y su gestión de dependencias permiten omitir versiones en muchos starters y librerías compatibles. Esa omisión no es falta de precisión; significa que la versión se resuelve mediante una política centralizada.

La ventaja pedagógica es importante: al añadir `spring-boot-starter-web`, no necesitamos elegir manualmente versiones independientes para Spring MVC, Tomcat, Jackson, logging y decenas de artefactos relacionados. Spring Boot proporciona un conjunto probado de versiones coherentes.

Dependencia directa y dependencia transitiva

Si nuestro POM declara A y A necesita B, Maven puede descargar B aunque no figure directamente en nuestro `<dependencies>`. B es transitiva.

Esto explica por qué `dependency:tree` es tan importante. Permite responder preguntas como:

- ¿por qué está esta librería en el classpath?;
- ¿qué dependencia la introdujo?;
- ¿qué versión terminó resolviéndose?;
- ¿hay exclusiones?;
- ¿existe una convergencia de versiones inesperada?

Pregunta

Si una clase está disponible aunque no hayas declarado su librería directamente, ¿es correcto asumir que siempre estará disponible?

Respuesta razonada

No. Puede estar llegando de forma transitiva. Si la dependencia que la introduce cambia o deja de incluirla, nuestro código puede romperse. Cuando una librería forma parte explícita del contrato de nuestro código, conviene entender de dónde procede y declarar las dependencias de manera consciente.

Resumen del Punto 0.2

- Initializr crea una base reproducible, no la lógica de negocio.
 - Cada campo del formulario tiene consecuencias en el proyecto.
 - El POM describe construcción y dependencias.
 - El parent de Spring Boot gestiona un conjunto coherente de versiones.
 - El Wrapper reduce diferencias entre equipos.
 - Un 404 inicial puede ser perfectamente correcto.
-

Punto 0.3 - Estructura estándar de un proyecto Spring Boot

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Reconocer la estructura Maven estándar.
2. Distinguir código de producción, recursos y tests.
3. Explicar el papel de la clase principal.
4. Entender el alcance del escaneo de componentes.
5. Explicar `application.properties`.
6. Entender qué es un controlador REST básico.
7. Razonar el flujo de una petición HTTP hasta un método Java.

Bloque 1 - Vista general

Una estructura mínima puede verse así:

```

mi-proyecto/
|-- .mvn/
|-- mvnw
|-- mvnw.cmd
|-- pom.xml
|-- src/
|   |-- main/
|   |   |-- java/
|   |   |   |-- es/mecd/demo/miproyecto/
|   |   |   |   |-- MiProyectoApplication.java
|   |   |-- resources/
|   |   |   |-- application.properties
|   |-- test/
|   |   |-- java/
|   |   |   |-- es/mecd/demo/miproyecto/
|   |   |   |   |-- MiProyectoApplicationTests.java
|-- target/
# aparece al construir; no se versiona

```

Spring Boot no exige de forma absoluta que cada proyecto tenga exactamente esta estructura, pero Maven y Spring trabajan especialmente bien cuando seguimos sus convenciones.

1.1 Por qué importa el paquete de la clase principal

@SpringBootApplication incluye escaneo de componentes desde el paquete donde está la clase principal hacia sus subpaquetes.

Si MiProyectoApplication está en:

```
es.mecd.demo.miproyecto
```

entonces un controlador en:

```
es.mecd.demo.miproyecto.controller
```

queda dentro del árbol natural de escaneo.

Un controlador colocado en un paquete hermano fuera de ese árbol puede no ser detectado automáticamente.

1.2 Fuentes frente a artefactos generados

Una distinción fundamental:

```

src/      -> lo que escribimos y versionamos
target/   -> lo que Maven genera

```

target/ puede contener clases compiladas, recursos copiados, resultados intermedios, informes y el JAR final.

Borrar target/ no borra el código fuente. Maven puede reconstruirlo. Por eso normalmente no se versiona.

1.3 Paquetes Java y carpetas

El package:

```
package es.mecd.demo.miproyecto.controller;
```

corresponde a una ruta parecida a:

```
src/main/java/es/mecd/demo/miproyecto/controller/
```

El paquete forma parte del nombre completo de la clase y evita colisiones.

1.4 Source sets de Maven

Maven reconoce por convención:

```
src/main/java
src/main/resources
src/test/java
src/test/resources
```

Producción y test quedan separados desde el principio.

Pregunta

¿Qué pasaría si colocáramos el controlador en `es.mecd.otro.controller` mientras la clase principal permanece en `es.mecd.demo.miproyecto`?

Respuesta razonada

Por defecto, el escaneo iniciado por `@SpringBootApplication` no recorrería ese paquete hermano. La clase compilaría, pero Spring podría no registrarla como bean; entonces su endpoint no existiría. Podríamos personalizar el escaneo, pero en un proyecto nuevo es más claro mantener la clase principal en un paquete raíz común.

Bloque 2 - Archivos de la raíz

`pom.xml`

Define el proyecto Maven, dependencias y plugins.

`.gitignore`

Evita versionar salidas generadas y configuración local del IDE. Por ejemplo `target/`, `.idea/` o ficheros temporales.

Maven Wrapper

`mvnw`, `mvnw.cmd` y `.mvn/wrapper/` permiten ejecutar el build con una versión controlada de Maven.

`target/`

Es un directorio generado. Contendrá clases compiladas, resultados de test, JAR y otros productos del build. Se puede borrar y regenerar.

`HELP.md` y otros archivos opcionales de Initializr

Dependiendo de la versión y opciones de Spring Initializr, el proyecto puede incluir documentación auxiliar como `HELP.md`. No forma parte del runtime de Spring Boot y su presencia o ausencia no altera la aplicación. Es una ayuda para la persona que abre el proyecto.

Esto ilustra una distinción que utilizaremos durante todo el curso:

- hay archivos que **construyen o configuran la aplicación**;
- hay archivos que **ayudan al desarrollo**;
- y hay archivos **generados** que no deben versionarse.

No debemos atribuir la misma importancia a todos sólo porque aparezcan en la raíz.

El ciclo Maven completo y qué directorio modifica

Las fases que verás con más frecuencia forman parte de un ciclo mayor:

validate -> compile -> test -> package -> verify -> install -> deploy

- validate: comprueba que el proyecto tiene una estructura utilizable;
- compile: compila producción hacia target/classes;
- test: compila y ejecuta tests;
- package: genera el artefacto, normalmente un JAR;
- verify: ejecuta verificaciones posteriores al empaquetado si hay plugins configurados;
- install: copia el artefacto al repositorio Maven local;
- deploy: publica el artefacto en un repositorio remoto configurado.

Al pedir una fase, Maven ejecuta las anteriores necesarias. Por eso package no significa «sólo compimir»: antes habrá compilado y ejecutado tests, salvo que se hayan desactivado explícitamente.

clean pertenece a otro ciclo y elimina target. Es útil para comprobar que una construcción no depende accidentalmente de restos de compilaciones anteriores.

Pregunta

¿Por qué no deberíamos editar manualmente clases dentro de target/classes?

Respuesta razonada

Porque son productos derivados. En el siguiente clean o compile pueden desaparecer o ser sustituidos por lo generado desde src/main. La fuente de verdad del código es src, no la salida del compilador.

Bloque 3 - src/main/java y src/main/resources

Código Java

src/main/java contiene código de producción.

Comenzaremos con una organización muy sencilla:

```
es.mecd.demo.miproyecto
`-- controller
```

Más adelante aparecerán servicios, repositorios, DTO, entidades, seguridad y otras áreas, pero no necesitamos anticiparlas todas en M0.

Recursos

src/main/resources contiene recursos que se incorporan al classpath de la aplicación. El archivo central en este momento es:

```
spring.application.name=mi-proyecto
```

application.properties permite externalizar configuración: puertos, logs, conexiones, perfiles, seguridad y muchas otras opciones que iremos introduciendo cuando sean necesarias.

Configuración externalizada

La idea importante es separar **código** de **configuración**. Un puerto, una URL de base de datos o un nivel de log no deberían requerir recompilar lógica de negocio si pueden expresarse como configuración.

application.properties, YAML y perfiles

Spring Boot permite expresar configuración tanto en `application.properties` como en `application.yml`. En este curso utilizaremos principalmente `.properties` para que las primeras prácticas mantengan una sintaxis sencilla, pero debes reconocer ambas posibilidades.

Una propiedad como:

```
server.port=8081
```

cambia el puerto sin recompilar el código. Esa separación entre código y configuración se vuelve crítica cuando una misma aplicación debe ejecutarse en desarrollo, test y producción.

Los perfiles permiten especializar valores mediante archivos como:

```
application-dev.properties
application-test.properties
application-prod.properties
```

No vamos a utilizar todavía perfiles para resolver problemas de negocio; se estudiarán con más profundidad cuando el curso introduzca configuración avanzada. Aquí basta con entender la arquitectura: el código permanece y cambia la configuración activa.

Advertencia importante: configuración externalizada no significa que sea correcto guardar secretos en Git. Contraseñas, tokens y claves deben venir de mecanismos apropiados, por ejemplo variables de entorno o gestores de secretos, cuando lleguemos a escenarios que los necesiten.

Pregunta

¿Qué ventaja tendrá, más adelante, poder utilizar `application-dev.properties` y `application-prod.properties`?

Respuesta razonada

Permitirá mantener el mismo código y variar configuración por entorno: base de datos, logging, URLs externas o credenciales gestionadas externamente. Así evitamos bifurcar el código sólo porque desarrollo y producción utilicen infraestructuras distintas.

Bloque 4 - src/test/java

El código de test está separado del código de producción. Maven compila y ejecuta ambos con classpaths relacionados pero distintos.

Un test inicial típico es:

```
@SpringBootTest
class MiProyectoApplicationTests {

    @Test
    void contextLoads() {
    }
}
```

Aunque el método esté vacío, `@SpringBootTest` hace que el test intente crear el contexto de Spring. Si la configuración principal está rota, el test puede fallar antes de ejecutar cualquier lógica de negocio.

Más adelante aprenderemos pruebas mucho más específicas. En M0 nos interesa primero entender que un build serio puede ejecutar pruebas de forma automática.

Pregunta

¿Por qué el código de producción no debería depender de clases que sólo existen en `src/test/java`?

Respuesta razonada

Porque el artefacto de producción se construye sin incluir las clases de test como dependencias de runtime. Una dependencia de producción hacia código de test rompería el empaquetado o requeriría convertir una ayuda de test en parte artificial de la aplicación. La dirección correcta es que los tests dependan del código que prueban.

4.1 @SpringBootApplication como anotación compuesta

Para M0 basta con usarla, pero conviene saber qué concentra conceptualmente:

- configuración Spring;
- auto-configuración;
- escaneo de componentes.

Puedes recordarlo como:

@SpringBootApplication
~ configuración + auto-configuración + component scan

4.2 Por qué importa la ubicación de la clase principal

Si MiProyectoApplication está en es.mecd.demo.miproyecto, Spring busca componentes en ese paquete y descendientes.

Por eso es.mecd.demo.miproyecto.controller funciona de forma natural.

Una clase @RestController colocada fuera de ese árbol puede no ser detectada. Este detalle explica muchos 404 aparentemente misteriosos.

Bloque 5 - Primer controlador REST

Usaremos:

```
package es.mecd.demo.miproyecto.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SaludoController {

    @GetMapping("/hola")
    public String saludar() {
        return construirMensaje("Ministerio de Educación");
    }

    @GetMapping("/adios")
    public String despedir() {
        return "Adiós, Ministerio de Educación";
    }

    private String construirMensaje(String destinatario) {
        return "Hola, " + destinatario;
    }
}
```

5.1 @RestController

Marca la clase como un componente web cuyos métodos pueden manejar peticiones y escribir directamente el valor de retorno en la respuesta HTTP.

En este momento nuestros métodos devuelven `String`. Más adelante devolveremos DTO y Spring utilizará Jackson para serializarlos a JSON.

5.2 @GetMapping

Asocia un método Java con peticiones HTTP GET para una ruta.

```
@GetMapping("/hola")
```

significa: cuando llegue una petición GET a `/hola`, este método es candidato a atenderla.

5.3 Flujo de una petición

Para:

```
curl http://localhost:8080/hola
```

el flujo simplificado es:

1. curl abre una conexión HTTP al puerto 8080.
2. Tomcat recibe la petición.
3. Spring MVC la procesa.
4. El mecanismo de mapeo encuentra GET `/hola`.
5. Spring invoca `SaludoController.saludar()`.
6. El método llama a `construirMensaje(...)`.
7. El `String` devuelto se escribe en la respuesta HTTP.
8. curl muestra el cuerpo.

El método privado está aquí también por una razón didáctica: en el siguiente punto podremos observar con un depurador la diferencia entre **Step Over** y **Step Into** sin necesitar todavía DTO, validaciones o servicios de módulos posteriores.

Pregunta

¿Qué método HTTP y qué ruta tendríamos que usar para obtener la despedida?

Respuesta razonada

GET `/adios`, porque `@GetMapping("/adios")` asocia esa combinación de método y ruta con `despedir()`. Una petición POST a la misma ruta no es equivalente: el método HTTP forma parte del contrato.

Ampliación conceptual - Estructura estándar, classpath y empaquetado

La estructura Maven no es sólo una convención estética. Cada árbol participa de manera distinta en el build.

src/main/java

Contiene código de producción. Maven lo compila durante `compile` y sus `.class` terminan formando parte del artefacto.

src/main/resources

Contiene recursos de producción. Maven los copia al classpath del resultado. Por eso `application.properties` queda accesible a Spring Boot sin convertirse en una clase Java.

src/test/java

Contiene código de test. Se compila después del código principal, dispone del classpath de test y no se incluye como código de producción en el JAR final.

src/test/resources

Cumple para los tests el papel que `src/main/resources` cumple para producción. Permite tener configuraciones y datos específicos de pruebas sin contaminar el artefacto final.

target

Es salida generada. Puede contener:

- clases compiladas;
- recursos copiados;
- clases de test;
- informes de Surefire;
- JAR;
- metadatos intermedios.

Debe poder borrarse y regenerarse. Por eso no es un lugar correcto para guardar trabajo manual.

Qué ocurre en package

De forma simplificada:

fuentes -> compile -> tests -> resources -> package -> JAR

En un proyecto Spring Boot, el plugin puede reempaquetar el JAR para incluir una estructura ejecutable con dependencias y lanzador. Eso permite:

```
java -jar ...
```

sin construir manualmente un classpath con decenas de JAR.

Pregunta

¿Por qué `target/` se ignora normalmente en Git aunque contenga el JAR que acabamos de construir?

Respuesta razonada

Porque es un resultado derivable. El repositorio debe guardar las fuentes y la configuración necesarias para reconstruirlo. Versionar artefactos intermedios o binarios generados introduce ruido, conflictos y riesgo de que el binario deje de corresponder al código fuente.

Ampliación conceptual - Escaneo de componentes y ubicación del paquete raíz

`@SpringBootApplication` combina varias ideas. Una de ellas es el escaneo de componentes a partir del paquete donde vive la clase principal.

Si la clase principal está en:

```
es.mecd.demo.miproyecto
```

una estructura natural es:

```
es.mecd.demo.miproyecto.controller  
es.mecd.demo.miproyecto.service  
es.mecd.demo.miproyecto.repository  
es.mecd.demo.miproyecto.config
```

El árbol descendente queda bajo el alcance habitual del component scan.

Si, en cambio, colocamos un controlador en un paquete hermano como:

```
es.mecd.demo.otro.controller
```

no debemos asumir que será descubierto automáticamente por la aplicación cuya raíz está en `es.mecd.demo.miproyecto`.

Este detalle conecta estructura de paquetes con comportamiento observable: un controlador perfectamente compilado puede no registrar ningún endpoint si Spring no lo descubre.

Pregunta

¿Por qué este tipo de error puede resultar engañoso para un principiante?

Respuesta razonada

Porque Java compila la clase y el IDE no muestra necesariamente ningún error. El problema aparece en runtime: el bean no existe en el contexto y la ruta devuelve 404. Es un buen ejemplo de diferencia entre corrección de compilación y configuración del framework.

Resumen del Punto 0.3

- Maven separa producción, recursos y tests mediante convenciones claras.
 - La clase principal debe situarse en una raíz de paquetes adecuada.
 - `@SpringBootApplication` inicia configuración, auto-configuración y escaneo.
 - Los recursos se integran en el classpath.
 - Un controlador REST convierte una ruta HTTP en una llamada a código Java.
 - La estructura no es decoración: condiciona compilación, escaneo, tests y empaquetado.
-

Punto 0.4 - Ciclo de trabajo diario: arrancar, leer logs, parar, depurar, testear y probar APIs

Objetivos de aprendizaje

Al finalizar este punto serás capaz de:

1. Explicar qué ocurre internamente al arrancar Spring Boot.
2. Reconocer las líneas importantes de un log de arranque.
3. Parar una aplicación de forma normal y distinguirlo de una terminación forzada.
4. Utilizar breakpoints y las operaciones fundamentales del depurador.
5. Ejecutar tests desde IDE y terminal.
6. Utilizar `curl` para inspeccionar una API.
7. Diagnosticar problemas habituales del ciclo diario.
8. Traducir los mismos conceptos entre IntelliJ, Eclipse y VS Code.

Bloque 1 - Qué ocurre al arrancar Spring Boot

La aplicación empieza como cualquier programa Java: la JVM invoca `main`.

```
public static void main(String[] args) {  
    SpringApplication.run(MiProyectoApplication.class, args);  
}
```

A partir de `SpringApplication.run(...)` se inicia una secuencia de alto nivel:

1. Se prepara y crea un `ApplicationContext`.
2. Se recopilan fuentes de configuración: propiedades, variables de entorno, argumentos, etc.
3. Se analiza la clase principal y se aplica la configuración asociada a `@SpringBootApplication`.
4. Se escanean componentes en el árbol de paquetes.
5. Se registran definiciones de beans.
6. Se crean beans y se resuelven dependencias.

7. Se evalúan condiciones de auto-configuración según clases y propiedades disponibles.
8. Al estar presente Spring Web, se crea la infraestructura web y se prepara el servidor embebido.
9. Tomcat empieza a escuchar en el puerto configurado, 8080 por defecto.
10. Se ejecutan callbacks de arranque como CommandLineRunner si existen.
11. El proceso permanece vivo esperando peticiones.

No todo ocurre estrictamente en una sola línea temporal simple; Spring dispone de varias fases internas. Para empezar, este modelo nos permite relacionar síntomas con etapas: un fallo al crear un bean ocurre antes de que el servidor quede listo; un conflicto de puerto aparece cuando el servidor intenta enlazar su socket.

Arrancar mediante Maven

```
./mvnw spring-boot:run
```

Maven utiliza el plugin de Spring Boot para ejecutar la aplicación durante desarrollo.

Arrancar el JAR

Primero:

```
./mvnw package
```

Después:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar
```

En este caso ejecutamos el artefacto empaquetado, un escenario más parecido a despliegue.

1.1 Las etapas del arranque con algo más de precisión

Cuando Spring Boot arranca, no sólo “levanta Tomcat”. Conviene distinguir:

Preparación del entorno. Se reúnen propiedades y argumentos.

Creación del contexto. Se prepara el contenedor de Spring.

Descubrimiento y registro. Se procesan configuraciones y componentes.

Creación de beans. Se instancian componentes y resuelven dependencias.

Auto-configuración. Se activan configuraciones según classpath, beans y propiedades.

Inicialización web. Se prepara Spring MVC y el servidor embebido.

Publicación del puerto. Tomcat enlaza el puerto y queda listo.

Callbacks posteriores. Se ejecutan runners si existen.

Esta clasificación permite diagnosticar: un error creando un bean y un puerto ocupado pertenecen a etapas distintas.

1.2 Tres formas de ejecutar y qué valida cada una

Desde el IDE. Valida la configuración local y facilita depuración.

Con spring-boot:run. Valida Maven y el plugin Spring Boot.

Con java -jar. Valida el artefacto ya empaquetado.

Las tres son útiles porque responden a preguntas distintas.

1.3 Argumentos desde línea de comandos

Puedes cambiar configuración sin editar código:

```
java -jar target/mi-proyecto-0.0.1-SNAPSHOT.jar --server.port=8081
```

Esto anticipa una idea clave de Spring Boot: la configuración puede externalizarse.

Pregunta

¿Qué diferencia conceptual hay entre `./mvnw spring-boot:run` y `java -jar ...`?

Respuesta razonada

En el primer caso Maven y el plugin participan directamente en la ejecución desde el proyecto. En el segundo ejecutamos el JAR ya empaquetado mediante la JVM. Ambos pueden iniciar la misma aplicación, pero ejercitan caminos distintos: uno depende del proyecto Maven y otro demuestra que el artefacto distribuible es autónomo.

Bloque 2 - Leer los logs

Una línea de log suele contener información como:

- fecha y hora;
- nivel (INFO, WARN, ERROR, etc.);
- identificador de proceso;
- hilo;
- logger o clase;
- mensaje.

Al arrancar nos interesan especialmente mensajes equivalentes a:

```
Tomcat started on port 8080 (http)
Started MiProyectoApplication in ... seconds
```

La redacción exacta puede cambiar entre versiones. Lo importante es reconocer las evidencias: servidor iniciado y contexto completado.

Niveles

- TRACE: detalle extremo.
- DEBUG: información útil para diagnóstico durante desarrollo.
- INFO: funcionamiento normal relevante.
- WARN: situación sospechosa o potencialmente problemática.
- ERROR: fallo.

No debemos confundir “muchos logs” con “mejor diagnóstico”. Aumentar a DEBUG/TRACE globalmente puede ocultar la señal entre miles de líneas. Es preferible activar detalle para paquetes concretos cuando exista una hipótesis.

Ejemplo temporal:

```
logging.level.es.mecd.demo=DEBUG
logging.level.org.springframework.web=DEBUG
```

Después de la investigación conviene retirar el cambio si no forma parte de la configuración deseada.

Problemas reconocibles en logs

Puerto ocupado. El servidor no puede enlazar 8080. Hay que detener el otro proceso o elegir otro puerto.

Bean ausente. Puede deberse a falta de anotación, paquete fuera del escaneo o configuración condicional no satisfecha.

Dependencia circular. Dos componentes dependen uno del otro de manera que el contenedor no puede construirlos según el modelo permitido.

ClassNotFoundException / NoClassDefFound. Puede existir una dependencia ausente o una diferencia entre classpaths.

Excepción de aplicación. El stack trace muestra la cadena de llamadas. Conviene localizar el primer punto relevante de nuestro código, no leer sólo la última línea.

2.1 Anatomía de una línea de log

Un log puede parecerse a:

```
2026-09-12T18:00:00.123+02:00 INFO 12345 --- [main] e.m.d.m.MiProyectoApplication : Started ...
```

Puedes leer:

- fecha/hora;
- nivel;
- PID;
- hilo;
- logger;
- mensaje.

Durante el arranque predomina main. Durante peticiones aparecerán hilos de trabajo del servidor.

2.2 Cómo leer un stack trace

Cuando aparece una excepción:

1. identifica el tipo;
2. busca Caused by: si existe;
3. localiza la primera línea de tu paquete;
4. relaciona clase y línea con tu código;
5. revisa la cadena de llamadas;
6. busca la causa raíz.

No es necesario comprender todavía todas las clases internas del framework para extraer información útil.

2.3 WARN no significa automáticamente “aplicación rota”

Un WARN merece atención, pero puede coexistir con un arranque correcto.

La evidencia debe combinar:

logs + proceso vivo + respuesta funcional

Pregunta

¿Qué dos evidencias buscarías antes de afirmar que la aplicación ha arrancado correctamente?

Respuesta razonada

Una evidencia del servidor web escuchando y otra de que el ApplicationContext ha completado el arranque. Si además una petición a /hola devuelve la respuesta esperada, obtenemos una verificación funcional externa y no sólo una lectura de logs.

Bloque 3 - Parar correctamente

En una terminal, Ctrl+C envía una señal de interrupción al proceso. La JVM y Spring pueden ejecutar su secuencia normal de cierre: parar servidor, cerrar contexto y liberar recursos.

Una terminación forzada como:

```
kill -9 <PID>
```

no ofrece esa oportunidad. Debe reservarse para procesos que no responden.

En Windows pueden utilizarse herramientas como tasklist y taskkill para localizar y terminar procesos.

Para descubrir quién usa el puerto 8080:

Linux/macOS:

```
lsof -i :8080
```

Windows:

```
netstat -ano | findstr :8080
```

3.1 Parada ordenada frente a terminación forzada

Cuando pulsas Stop en el IDE o Ctrl+C en una terminal, la aplicación recibe una señal de terminación y tiene oportunidad de cerrar recursos de forma ordenada. El servidor deja de aceptar trabajo, el contexto de Spring se cierra y los recursos administrados pueden liberar conexiones o ejecutar lógica de destrucción.

Una terminación forzada, como kill -9 en sistemas Unix, no concede esa oportunidad. El sistema operativo elimina el proceso inmediatamente. Puede ser necesaria si un proceso está completamente bloqueado, pero no debe ser el mecanismo normal de parada.

Por eso el orden de preferencia es:

1. Stop del IDE o Ctrl+C;
2. terminación normal por PID si hace falta;
3. fuerza bruta sólo como último recurso.

3.2 Cómo demostrar que la aplicación realmente se ha parado

Que la consola deje de escribir no es suficiente. Puedes verificarlo intentando acceder al endpoint:

```
curl -i http://localhost:8080/hola
```

Si no hay otro proceso escuchando, la conexión fallará. También puedes comprobar el puerto con herramientas del sistema (netstat, ss, lsof, según plataforma). Esta comprobación es especialmente útil antes de diagnosticar un «Port 8080 already in use».

Pregunta

¿Qué diferencia hay entre Ctrl+C y una terminación forzada? ¿Por qué importa?

Respuesta razonada

Ctrl+C solicita terminar y permite normalmente un cierre ordenado. Una señal de terminación forzada detiene el proceso sin garantizar callbacks de cierre ni liberación ordenada de recursos por la aplicación. En desarrollo puede parecer equivalente, pero en sistemas con conexiones, buffers o trabajo pendiente la diferencia es importante.

Bloque 4 - Depuración paso a paso

4.1 Breakpoint

Un breakpoint ordena al depurador pausar cuando la ejecución llega a una ubicación concreta.

Con el programa detenido podemos:

- inspeccionar variables;
- ver la pila de llamadas;
- evaluar expresiones;
- avanzar una línea;
- entrar en una llamada;
- salir del método actual;
- continuar hasta el siguiente breakpoint.

4.2 Step Over, Step Into y Step Out

Supón:

```
public String saludar() {
    return construirMensaje("Ministerio de Educación");
}
```

Step Over ejecuta la llamada a `construirMensaje` como una unidad y se detiene en la siguiente línea disponible del contexto actual.

Step Into entra dentro de `construirMensaje` para ver sus instrucciones.

Step Out completa el resto del método actual y vuelve al método que lo llamó.

Continue/Resume reanuda la ejecución hasta el siguiente breakpoint o hasta terminar.

4.3 Atajos típicos

Acción	IntelliJ	Eclipse	VS Code
Step Over	F8	F6	F10
Step Into	F7	F5	F11
Step Out	Shift+F8	F7 / Step Return	Shift+F11
Resume/Continue	F9	F8	F5
Detener	Ctrl+F2	botón Stop / Ctrl+F2 según configuración	Shift+F5

Los atajos pueden variar por keymap o sistema operativo. Lo importante es dominar la operación, no memorizar una tecla como si fuera parte de Java.

4.4 Pila de llamadas y contexto actual

La **call stack** responde a “¿cómo hemos llegado hasta esta línea?”.

En una petición Spring MVC verás frames del framework y de tu código. Cambiar de frame permite inspeccionar variables de distintos niveles.

4.5 Watches y evaluación de expresiones

Durante una pausa puedes evaluar expresiones sin modificar el fuente.

Un *watch* mantiene visible una expresión mientras avanzas paso a paso.

4.6 Breakpoints condicionales

En métodos muy invocados, detenerse siempre puede resultar incómodo. Un breakpoint condicional sólo se activa cuando una expresión es verdadera.

Aprenderemos el mecanismo ahora y lo utilizaremos más cuando existan identificadores, colecciones y bucles.

4.7 Estrategia de depuración

Una secuencia útil:

1. reproducir el fallo;
2. elegir un primer punto observable;
3. comprobar entradas;
4. avanzar hasta el primer estado inesperado;
5. corregir la causa;
6. repetir la prueba.

Depurar no consiste en entrar en todo, sino en reducir el espacio de búsqueda.

4.8 Herramientas concretas que encontrarás en los IDEs

Los tres IDEs expresan los mismos conceptos con nombres ligeramente distintos.

IntelliJ IDEA ofrece las vistas Variables, Watches, Frames y Evaluate Expression. Un breakpoint se activa desde el margen; los breakpoints condicionales se editan desde las propiedades del punto de ruptura. Evaluate Expression permite ejecutar expresiones Java usando el contexto en pausa.

Eclipse organiza la información en la perspectiva Debug, con vistas Variables, Breakpoints, Debug y Expressions. Inspect/Display y la vista Expressions cumplen el papel de evaluación y vigilancia de valores.

VS Code muestra Variables, Watch, Call Stack y Breakpoints en Run and Debug, y dispone de Debug Console para evaluar expresiones durante una pausa.

Lo importante no es memorizar una interfaz concreta. Debes saber formular la intención:

- quiero detenerme aquí;
- quiero avanzar sin entrar;
- quiero entrar en este método;
- quiero volver al llamador;
- quiero vigilar esta expresión;
- quiero que el breakpoint sólo se active si se cumple una condición;
- quiero saber por qué cadena de llamadas he llegado hasta aquí.

Cuando conoces la intención, cambiar de IDE es un problema de localizar el botón equivalente, no de reaprender la depuración.

Pregunta

¿Cuándo usarías Step Into en vez de Step Over?

Respuesta razonada

Cuando sospecho que el comportamiento incorrecto está dentro del método llamado y necesito observar su lógica. Si la llamada está suficientemente probada o no me interesa su interior, Step Over evita entrar y mantiene la depuración centrada en el nivel actual.

Bloque 5 - Tests en el ciclo diario

Desde terminal:

```
./mvnw test
```

Ejecutar tests desde Maven es importante porque es reproducible fuera del IDE y es el mismo tipo de operación que realizará un pipeline de CI.

En este módulo tenemos dos clases con objetivos distintos:

- `MiProyectoApplicationTests`: verifica que el contexto puede cargarse.
- `SaludoControllerTest`: verifica directamente el comportamiento Java del controlador sin introducir todavía `MockMvc`.

La segunda prueba es deliberadamente simple:

```
assertThat(controller.saludar())  
    .isEqualTo("Hola, Ministerio de Educación");
```

Más adelante aprenderemos tests HTTP y tests aislados de capas; en M0 queremos dominar primero el ciclo ejecutar -> observar -> corregir -> repetir.

Pregunta

¿Qué ventaja tiene poder ejecutar los mismos tests desde terminal además de desde el IDE?

Respuesta razonada

La terminal elimina dependencia de una configuración gráfica concreta y permite automatizar. Un servidor de CI puede clonar el repositorio y ejecutar exactamente el mismo comando. Si una prueba sólo funciona cuando hacemos clic en una configuración privada del IDE, no es suficientemente reproducible.

Bloque 6 - curl como instrumento de observación HTTP

`curl` permite enviar peticiones desde terminal y ver la respuesta sin depender de un navegador.

Petición básica:

```
curl http://localhost:8080/hola
```

Incluir cabeceras de respuesta:

```
curl -i http://localhost:8080/hola
```

Modo detallado de conexión y protocolo:

```
curl -v http://localhost:8080/hola
```

Algunas opciones que utilizaremos durante el curso:

Opción	Propósito
-X	indicar explícitamente un método HTTP
-H	añadir una cabecera
-d	enviar datos en el cuerpo
-i	incluir cabeceras de respuesta en la salida
-v	mostrar detalle de la conversación HTTP y conexión
-b	enviar cookies
-u	autenticación básica
-o	guardar respuesta en un fichero
-s	reducir salida de progreso

No necesitamos usar todas en M0; las introducimos para reconocerlas cuando aparezcan.

6.1 Método, URL, cabeceras y cuerpo

Una petición HTTP se puede pensar como:

método + URL + cabeceras + cuerpo opcional

Ejemplo de sintaxis que utilizaremos en módulos posteriores:

```
curl -X POST \
-H "Content-Type: application/json" \
-d '{"nombre":"Ana"}' \
http://localhost:8080/ejemplo
```

6.2 Redirecciones

Para seguir redirecciones:

```
curl -L http://localhost:8080/alguna-ruta
```

6.3 Windows PowerShell

Si curl parece comportarse de forma extraña, prueba explícitamente:

```
curl.exe -i http://localhost:8080/hola
```

6.4 Guardar y silenciar salidas

```
curl -o respuesta.txt http://localhost:8080/hola
curl -s http://localhost:8080/hola
```

Las opciones pueden combinarse; eso es más importante que memorizarlas aisladas.

6.5 Flags de curl que conviene reconocer desde el principio

No necesitamos memorizar todo curl, pero sí entender qué dimensión de la petición o respuesta modifica cada opción.

Opción	Qué hace	Cuándo ayuda
-i	incluye cabeceras de respuesta	comprobar status, Content-Type, Location, etc.
-v	muestra conversación detallada	diagnosticar conexión, cabeceras y redirecciones
-X	fija explícitamente el método	pruebas de POST/PUT/PATCH/DELETE
-H	añade una cabecera	Content-Type, Authorization, cabeceras propias
-d	envía datos	cuerpos JSON o formularios
-b	envía cookies	reproducir sesiones/estado del cliente
-u	construye autenticación básica	ejercicios de seguridad posteriores
-o	guarda el cuerpo en archivo	respuestas grandes/binarias
-s	reduce salida de progreso	scripts y automatización
-L	sigue redirecciones	observar el destino final de respuestas 3xx

Algunas opciones se usarán mucho más adelante. Presentarlas ahora sirve para que, cuando aparezcan, el comando no parezca una secuencia arbitraria de letras.

En PowerShell existe una peculiaridad histórica: curl puede resolverse a un alias/comando distinto según la versión del entorno. Si necesitas garantizar el ejecutable de cURL, utiliza curl.exe y comprueba con Get-Command curl qué se está resolviendo.

Pregunta

¿Qué diferencia hay entre curl -i y curl -v?

Respuesta razonada

-i añade las cabeceras de la **respuesta** a la salida que normalmente mostraría sólo el cuerpo. -v activa una traza más detallada de la operación, incluyendo información de conexión y cabeceras intercambiadas. Para comprobar rápidamente un Content-Type o status puede bastar -i; para diagnosticar la conversación completa es más útil -v.

Bloque 7 - El mismo flujo en tres IDEs

IntelliJ IDEA

- Abrir la clase principal.
- Usar Run para ejecución normal y Debug para depuración.
- Los logs aparecen en la ventana Run/Debug.
- Los breakpoints se colocan en el margen del editor.
- Los tests se pueden ejecutar desde el icono junto a clase/método.
- Maven puede ejecutarse desde su ventana de herramientas o desde la terminal integrada.

Eclipse

- Ejecutar la clase principal con Run As / Java Application o configuración equivalente.
- Depurar con Debug As.
- Logs en Console.
- Breakpoints desde el margen.
- Tests con Run As / JUnit Test.
- Maven se integra mediante el soporte Maven del IDE.

VS Code

- Requiere las extensiones Java/Spring apropiadas.
- Puede ejecutar desde los enlaces Run/Debug sobre main, desde el panel de Spring Boot o desde configuraciones de depuración.
- Logs aparecen en terminal/debug console según el modo.
- Breakpoints se sitúan en el margen.
- Test Explorer permite ejecutar pruebas.
- La terminal integrada es especialmente útil para Maven y curl.

IntelliJ IDEA: flujo detallado

- El triángulo junto a main permite ejecutar.
- Debug arranca el mismo programa bajo depurador.
- **Run** muestra salida.
- **Debug** expone variables, stack, watches y breakpoints.
- **Run > Edit Configurations** permite revisar argumentos y entorno.
- Los tests pueden lanzarse desde el icono de margen.

Si Maven funciona en terminal pero IntelliJ muestra dependencias rojas, primero recarga el modelo Maven.

Eclipse: flujo detallado

- Run As > Java Application ejecuta la clase principal.
- Debug As > Java Application la ejecuta con depurador.
- **Console** muestra logs.
- La perspectiva Debug expone variables, breakpoints y stack.
- Run As > JUnit Test ejecuta tests.
- m2e mantiene la integración Maven.

VS Code: flujo detallado

Con las extensiones Java/Spring:

- Run / Debug aparecen sobre main;
- **Run and Debug** muestra Variables, Watch, Call Stack y Breakpoints;
- **Testing** ejecuta pruebas;
- la terminal integrada es natural para Maven y curl.

Mismo concepto, distintas etiquetas

Concepto	IntelliJ	Eclipse	VS Code
salida	Run / Debug	Console	Terminal / Debug Console
ejecutar	Run	Run As	Run
depurar	Debug	Debug As	Debug
pila	Frames	Debug view	Call Stack
watch	Watches	Expressions	Watch
tests	gutter / Run	JUnit	Testing

Arranque: equivalencias operativas

Acción	IntelliJ IDEA	Eclipse	VS Code
ejecutar clase principal	icono Run / Shift+F10 según keymap	Run As > Java Application / Ctrl+F11	enlace Run
depurar	Debug / Shift+F9 según keymap	Debug As > Java Application	enlace Debug
detener	Stop	Stop	Stop
editar configuración	Run/Debug Configurations	Run Configurations	launch configuration

Los atajos pueden variar con el sistema operativo o keymap. La guía los presenta como referencia, pero el concepto importante es la acción.

Depuración: equivalencias

Acción	IntelliJ IDEA	Eclipse	VS Code
Step Over	F8 habitual	F6	F10
Step Into	F7 habitual	F5	F11
Step Out/Return	Shift+F8	F7	Shift+F11
Resume	F9	F8	F5
evaluar expresión	Evaluate Expression	Inspect/Expressions	Debug Console / Watch

No memorices los números de tecla como conocimiento Spring. Aprende qué efecto necesitas y consulta el keymap si trabajas en otro IDE.

Ejecutar tests desde los tres IDEs

IntelliJ IDEA: el icono de ejecución junto a una clase o método JUnit permite lanzar una prueba concreta. También puedes ejecutar una carpeta de tests o usar la ventana Maven.

Eclipse: Run As > JUnit Test ejecuta la selección. La vista JUnit muestra pruebas verdes/rojas y permite navegar al fallo.

VS Code: la vista Testing y los enlaces sobre los métodos permiten ejecutar o depurar pruebas. Las extensiones Java proporcionan el runner.

En los tres casos debes poder volver a terminal y ejecutar:

```
./mvnw test
```

Si el IDE muestra verde pero Maven falla, la discrepancia es información: quizá el IDE está ejecutando un subconjunto, otra configuración o una JDK distinta.

Cómo leer un fallo de test

Un test fallido aporta varias capas de información:

1. nombre del test;
2. tipo de fallo o excepción;
3. valor esperado y valor real si es una aserción;
4. stack trace;
5. ubicación de la primera línea relevante de nuestro código.

No corrijas un test sólo para ponerlo verde si el comportamiento esperado era correcto. Primero decide cuál de las dos cosas está equivocada: la implementación o la expectativa.

Diagnóstico rápido del ciclo diario

Síntoma	Hipótesis inicial	Primera comprobación
no arranca	build/configuración/puerto	primer ERROR y causa raíz
arranca pero endpoint falla	mapping/URL/lógica	curl -i y breakpoint
breakpoint no salta	no estás en Debug o la ruta no pasa por ahí	modo de ejecución y URL
tests IDE sí, Maven no	entornos distintos	JDK y comando Maven
8080 ocupado	proceso anterior	localizar PID/puerto
cambio no aparece	proceso viejo/build/DevTools	recompilar/reiniciar y verificar PID

El objetivo no es tener una tabla para cada posible error, sino adquirir una disciplina: **clasifica la fase que falla antes de cambiar código**.

Pregunta

Si cambias de IntelliJ a VS Code, ¿cambia Spring Boot o cambia la interfaz con la que lanzas y depuras el mismo programa?

Respuesta razonada

Spring Boot y el bytecode de la aplicación no cambian por usar otro editor. Cambia la interfaz del IDE, sus atajos y su integración con JDK/Maven. Por eso conviene aprender primero conceptos como breakpoint, Step Into, test y proceso Maven; luego se traducen al botón concreto de cada herramienta.

Ampliación conceptual - El mismo ciclo de trabajo en cuatro interfaces

Para este curso vamos a tratar **consola, IntelliJ IDEA, Eclipse y VS Code** como cuatro interfaces distintas sobre las mismas operaciones fundamentales. El alumno debe reconocer la operación antes que el botón.

Construir

- Consola: Maven Wrapper.
- IntelliJ: ventana Maven o acciones de build.
- Eclipse: integración Maven/m2e o Maven Build.
- VS Code: terminal integrada y acciones Java/Maven proporcionadas por extensiones.

Ejecutar

- Consola: `spring-boot:run` o `java -jar`.
- IntelliJ: Run Configuration.
- Eclipse: Run As / Spring Boot App.
- VS Code: Run sobre main o Run and Debug.

Depurar

Los cuatro conceptos importantes son independientes del IDE:

- breakpoint;
- stack de llamadas;
- variables;
- control del flujo.

Los nombres Step Over, Step Into, Step Out y Resume aparecen con ligeras variaciones visuales, pero representan las mismas operaciones del depurador.

Testear

- Consola: `mvnw test`.
- IntelliJ: runner JUnit integrado.
- Eclipse: JUnit view.
- VS Code: Testing view / Java Test Runner.

El criterio de éxito no debe ser «sale verde en mi IDE», sino que el test tenga una aserción significativa y que el build reproducible también pase.

Probar HTTP

El navegador resulta cómodo para GET sencillos. `curl` es más general porque permite elegir método, cabeceras, cuerpo, autenticación, cookies y modo detallado. Más adelante aparecerán clientes especializados, pero aprender `curl` proporciona una referencia universal.

Pregunta

¿Por qué insistimos en aprender la operación conceptual antes que el botón concreto del IDE?

Respuesta razonada

Porque los menús cambian entre herramientas y versiones, mientras que compilar, ejecutar, depurar, testear y enviar una petición HTTP son operaciones estables. Entender el concepto permite trasladarse entre IntelliJ, Eclipse, VS Code, terminal, CI y servidores sin empezar de cero.

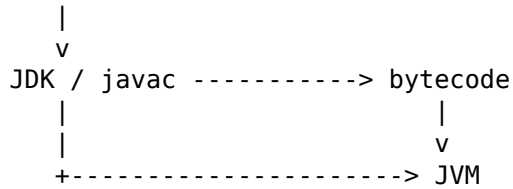
Resumen del Punto 0.4

- `main` es el punto de entrada y `SpringApplication.run` inicia el contexto Spring.
 - Los logs permiten distinguir etapas de arranque y diagnosticar fallos.
 - Un cierre normal no es equivalente a matar un proceso a la fuerza.
 - El depurador permite observar estado y flujo de ejecución.
 - Maven ofrece una forma reproducible de ejecutar tests.
 - `curl` hace visible el contrato HTTP.
 - Los IDEs cambian la interfaz, no los fundamentos.
-

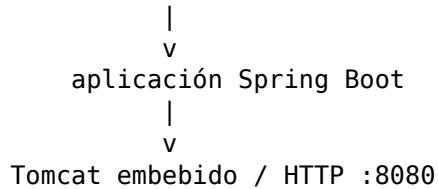
Síntesis del Módulo 0

Al terminar la teoría de este módulo deberías manejar un modelo mental completo del entorno:

Código Java



pom.xml ---> Maven ---> dependencias + compile + test + package



El IDE rodea este flujo con una interfaz productiva, pero el proyecto debe seguir siendo comprensible y ejecutable desde terminal. La parte práctica convertirá este modelo en acciones verificables: comprobar el entorno, compilar Java sin Spring, generar un proyecto, leer el POM, crear endpoints, ejecutar tests, empaquetar, arrancar el JAR, depurar y utilizar curl.